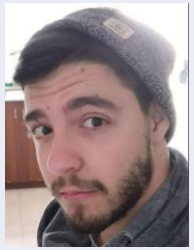
Course: Image Processing 31651
Micro Project 2
Usage Of Filtration

Photo	First Name	Last Name- First 4 Letters	4 Last Digits ID
	Niv	Kopo	2178
	Danny	Port	7352
	Daniel	Vais	7515

MP2. Abstract

- The project explores the application of convolution matrices in digital image processing, focusing primarily on two key operations: blurring and sharpening.
- Convolution matrices are utilized as powerful tools to achieve these effects efficiently.
- In addition, this project explores the application of Fast Fourier Transform (FFT) in the frequency domain for blurring and restoring images using Gaussian filters.
- Spatial Domain vs. Frequency Domain: In digital image processing, images are represented as a grid of pixel values in the spatial domain, where each pixel corresponds to a specific location and intensity.
- Fourier Transform is a mathematical operation that converts signals or functions from the spatial domain into the frequency domain.
- The project aims to enhance understanding of these techniques and their practical implications.

MP2- List Of Items To Be Discussed

MP2.11 – Introduction + VGA BMP Image Creation – Tim1.bmp	4
MP2.12 – Blur Image By Using Fast Gaussian Filters and Convolution	7
MP2.13 – Blur Image By Using FD Gaussian Filter and FFT	13
MP2.14 – Restore Blurred Images - Convolution	16
MP2.15 – Restore Blurred Images – FFT	21
MP2.16 – Restore Blurred Images – Results and Discussion	22
MP2.21 – TV Test BMP Image – Tim2.bmp	33
MP2.22 – Blur Image By Using Fast Gaussian Filters and Convolution	34
MP2.23 – Blur Image By Using FD Gaussian Filter and FFT	37
MP2.24 – Restore Blurred Images – Convolution	39
MP2.25 – Restore Blurred Images – FFT	40
MP2.26 – Restore Blurred Images – Results and Discussion	41
MP2.3 – Main Code	52
MP2.4 – Flow Charts	53
MP2.5 – What Did We Learn	62
MP2.6 – List of Sources – Bibliography	63

MP2.11. Introduction – Filtration By Convolution

- A kernel, convolution matrix is a small matrix used for blurring, sharpening, embossing, edge detection, and more.
- This is accomplished by doing a convolution between the kernel and the source image.
- Convolution is the process of adding each element of the image to its local neighbors, weighted by the kernel. This is related to a form of mathematical convolution.
- This segment of the project explores the application of Filtration By Convolution in the spatial domain for blurring and restoring images.

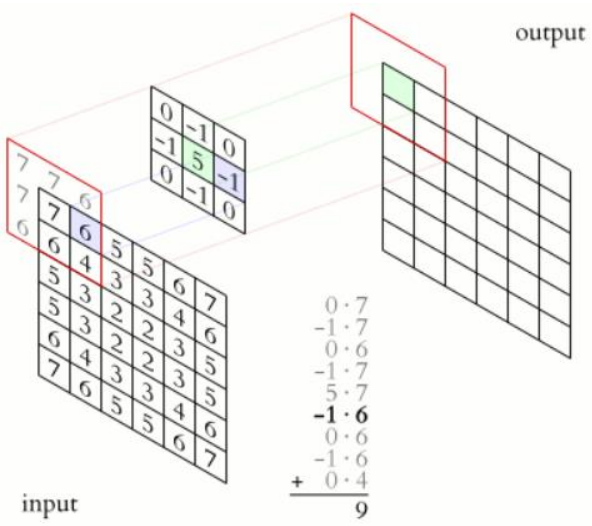


Fig 1: 2D Convolution Example

MP2.11. Introduction – Fast Fourier Transform (FFT)

- Fast Fourier Transform (FFT) is a fundamental tool in digital image processing, enabling efficient analysis, manipulation, and enhancement of images in the frequency domain.
- Its applications range from basic filtering operations to advanced techniques such as image restoration and compression.
- FFT is an algorithm for efficiently computing the Fourier Transform of a discrete signal or image. It dramatically reduces the computational complexity of the Fourier Transform, making it practical for real-time processing of digital signals, including images.
- This segment of the project explores the application of Fast Fourier Transform (FFT) in the frequency domain for blurring and restoring images.

MP2.11. VGA BMP Image Creation – Tim1.bmp

- First, we created BMP VGA Image (640x480) Tim1.bmp (Fig 2) By using MS Paint.
- The image contains 6 lines of the text: “Hello from 2178-7352-7515”
- Font size of every line is different (from small to large).

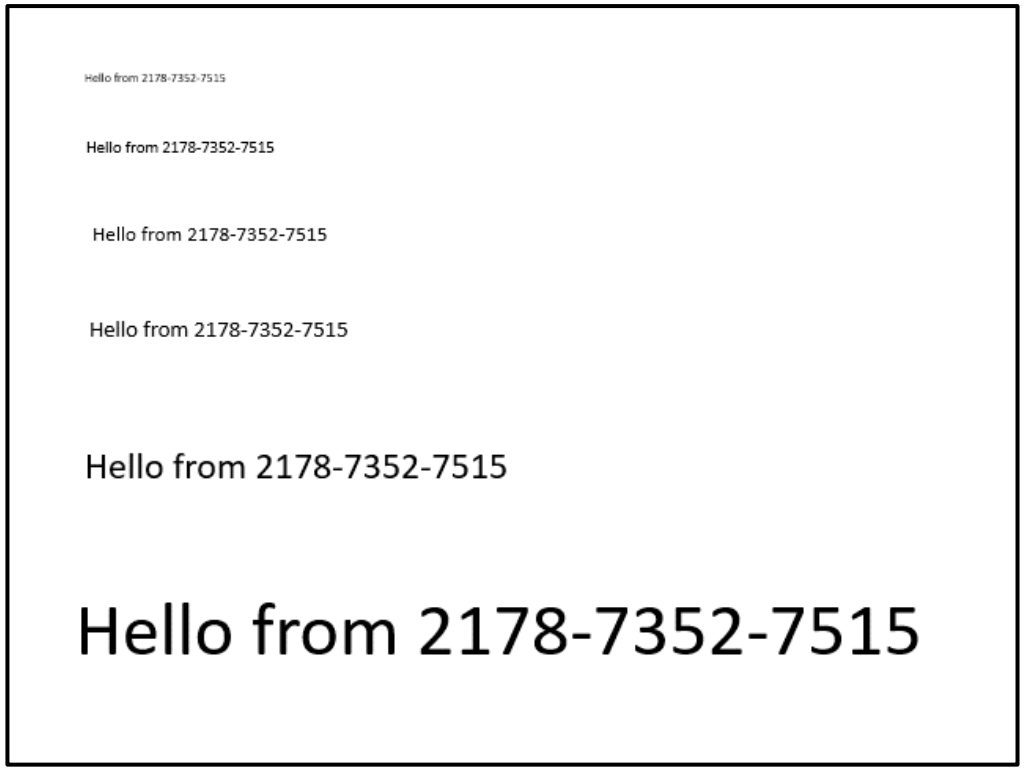


Fig 2: 6 Lines Text Image – Tim1.bmp

MP2.12. Blur Image By Using Fast Gaussian Filters and Convolution

- Fast Gaussian filters utilize approximation techniques to accelerate the blurring process while maintaining the Gaussian effect over the image.
- One common approach is to exploit the separability property of Gaussian filters, which states that a 2D Gaussian kernel can be decomposed into two 1D kernels (one for rows and one for columns).
- Instead of directly convolving the image with a 2D Gaussian kernel, the image is convolved separately along rows and columns with 1D Gaussian kernels.
- By applying two consecutive 1D convolutions instead of a single 2D convolution, fast Gaussian filters achieve a significant speedup in computation.
- Accuracy vs. Speed Tradeoff: While fast Gaussian filters offer computational efficiency, there might be a tradeoff in terms of accuracy compared to the exact Gaussian blur achieved by 2D convolution. However, in practice, the difference in quality may be negligible for many applications.

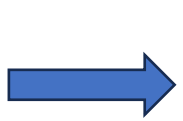
MP2.12. Blur Image By Using Fast Gaussian Filters and Convolution

- Our project involves working with two 2D filters, sized 5x5 (Fig 3) and 7x7 (Fig 4).
- The goal is to optimize these filters into fast Gaussian filters to enhance computational efficiency.
- The process includes decomposing the 2D filters into two 1D kernels for both rows and columns.
- This decomposition aims to accelerate the blurring process significantly while maintaining the desired Gaussian blur effect.
- The Resulted Fast Gaussian (1D) filters are sized 5 (Fig 5) and 7 (Fig 6).

1/273

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

Fig 3: 2D 5x5 Gaussian Filter



.061	.242	.383	.242	.061

Fig 5: 1D Fast Gaussian Filter- Size 5

1/1003

0	0	1	2	1	0	0
0	3	13	22	13	3	0
1	13	59	97	59	13	1
2	22	97	159	97	22	2
1	13	59	97	59	13	1
0	3	13	22	13	3	0
0	0	1	2	1	0	0

Fig 4: 2D 5x5 Gaussian Filter



.006	.061	.242	.383	.242	.061	.006

Fig 6: 1D Fast Gaussian Filter- Size 7

MP2.12. Blur Image By Using Fast Gaussian Filters and Convolution

- This code (Fig 7) applies a fast Gaussian filter convolution to an input image.
- It loads the input image from a file specified by inputFileName.
- Two different low-pass filter kernels, LPF1 and LPF2, are defined.
- The DoGaussianFiltration function is called twice.
  - First, with LPF1 and a parameter 2.
  - Second, with LPF2 and a parameter 3.
- Output filenames for the filtered images are constructed by appending suffixes to the input filename.
- The filtered images are saved with the constructed filenames.
- The function returns a vector containing the filenames of the filtered images.

```
145  // Applies 2 LPF filters to the given image and returns the
146  // file names of the blurred images (with added "LPF5c" and "LPF7c" suffixes)
147  vector<string> ApplyFastGaussianFilterConvolution(string inputFileName)
148  {
149      c2dByteGrayImage source(AddBMPExtention(inputFileName).c_str());
150      double LPF1[5] = { 0.061,0.242,0.383,0.242,0.061 };
151      double LPF2[7] = { 0.006,0.061,0.242,0.383,0.242,0.061,0.006 };
152      DoGaussianFiltration(source, LPF1, 2);
153      string LPF1Suffix = "LPF5c";
154      string LPF2Suffix = "LPF7c";
155      vector<string> outputFileNames =
156      {
157          inputFileName + LPF1Suffix,
158          inputFileName + LPF2Suffix
159      };
160      source.SaveToGrayBmpFile(AddBMPExtention(outputFileNames[0]).c_str());
161      source.Init(AddBMPExtention(inputFileName).c_str());
162      DoGaussianFiltration(source, LPF2, 3);
163      source.SaveToGrayBmpFile(AddBMPExtention(outputFileNames[1]).c_str());
164      return outputFileNames;
165  }
```

Fig 7: Code for applying fast gaussian

MP2.12. Blur Image By Using Fast Gaussian Filters and Convolution

- This function performs a Gaussian filtration on a 2D gray-scale image (Fig 8).
- It takes the image, a filter kernel, and half the size of the filter as input parameters. It initializes a temporary destination image with the same dimensions as the source image.
- The function iterates over the rows and columns of the source image. For each pixel, it convolves the filter kernel in the horizontal direction (x).
- The filtered pixel value is stored in the temporary destination image.
- After filtering in the horizontal direction, the source image is cleared.
- The function then iterates over the rows and columns of the temporary destination image.
- For each pixel, it convolves the filter kernel in the vertical direction (y).
- The filtered pixel value is stored back in the source image.

```
21  void DoGaussianFiltration(c2dByteGrayImage& source, double filter[], int filterHalfSize)
22  {
23      int numRows = source.NumberOfRows();
24      int numColumns = source.NumberOfColumns();
25      c2dByteGrayImage temp_dest(numRows, numColumns);
26      double summa;
27
28      // x direction
29      for (int row = 0; row < numRows; row++)
30      {
31          for (int column = filterHalfSize; column < numColumns - filterHalfSize; column++)
32          {
33              summa = 0;
34              for (int x = -filterHalfSize; x <= filterHalfSize; x++)
35              {
36                  summa = summa + filter[filterHalfSize + x] * (double)source.GetPixelValue(row, column + x);
37              } // of x
38              if (summa < 0)
39                  summa = 0;
40              if (summa > 255)
41                  summa = 255;
42              temp_dest.SetPixelValue(row, column, (tByte)(summa + 0.5));
43
44          } // of column
45      } // of row
46
47      // Clear Source !!!
48      source.Clear();
49
50      // y direction
51      for (int row = filterHalfSize; row < numRows - filterHalfSize; row++)
52      {
53          for (int column = 0; column < numColumns; column++)
54          {
55              summa = 0;
56              for (int y = -filterHalfSize; y <= filterHalfSize; y++)
57              {
58                  summa += filter[filterHalfSize + y] * (double)temp_dest.GetPixelValue(row + y, column);
59
60              } // of y
61              if (summa < 0) summa = 0;
62              if (summa > 255) summa = 255;
63              source.SetPixelValue(row, column, (tByte)(summa + 0.5));
64          } // of column
65      } // of row
66  }
```

Fig 8: Code for DoGaussianFiltration

MP2.12. Blur Image By Using Fast Gaussian Filters and Convolution

- Blurring is achieved by convolving the image with a blur kernel, which redistributes pixel values to create a smoother appearance.
- Implementation involves sliding the kernel over the image and computing the weighted sum of neighboring pixels.

- We applied Fast Gaussian filter (1D Filter) with size=5 over the original Tim1.bmp image (Fig 9) by using convolution.
- Applying the Fast Gaussian filter, will result in the image being smoothed or blurred (Fig 10).
- There are two lines that are hardly readable and another two line that are unreadable.
- Applying a Gaussian LPF can result in the image appearing darker because it reduces overall contrast.
- The resulting image will feature a black frame of 2 pixels, as the convolution is exclusively applied to valid positions for the kernel.
- Half the size of the kernel will be removed from the image boundaries

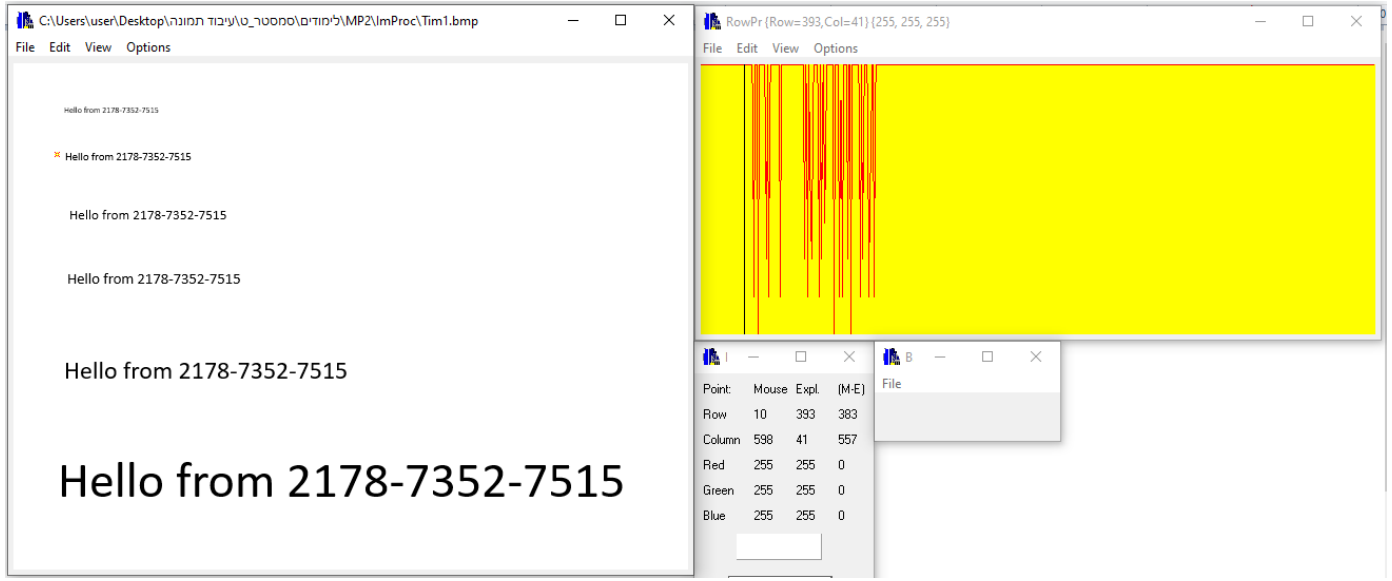


Fig 9: Profilers of Tim1.bmp

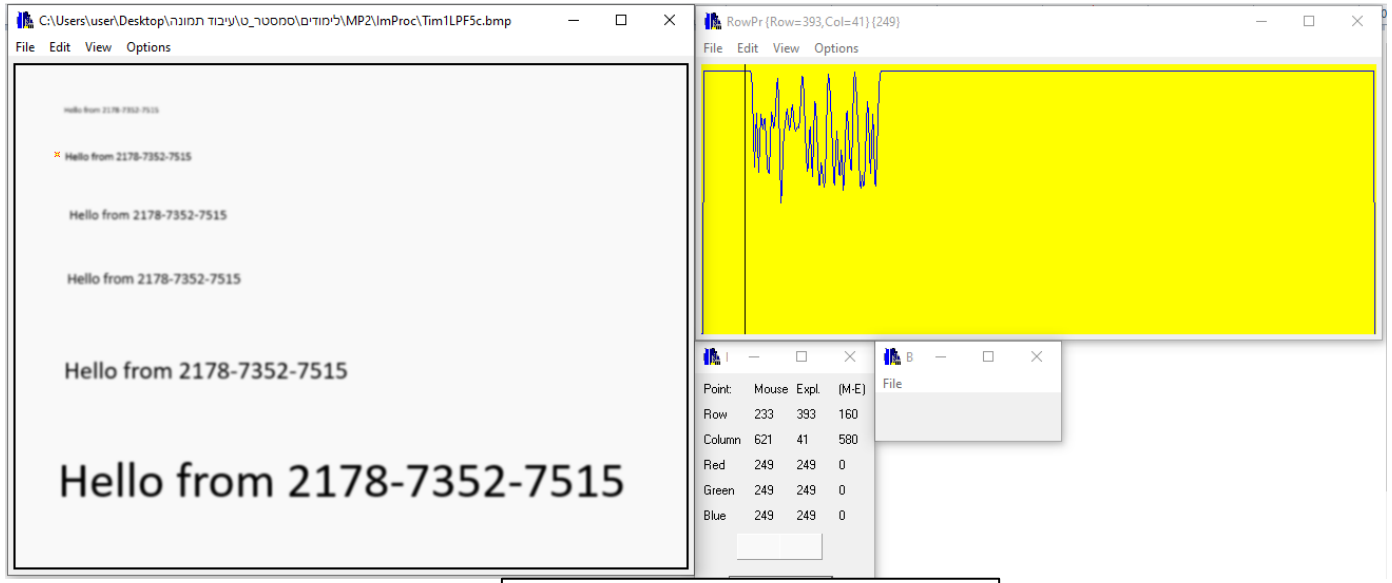


Fig 10: Profilers of Tim1LPF5c.bmp

MP2.12. Blur Image By Using Fast Gaussian Filters and Convolution

- We applied Fast Gaussian filter (1D Filter) with size=7 over the original Tim1.bmp image (Fig 11) by using convolution.
- Applying the Fast Gaussian filter, will result in the image being smoothed or blurred (Fig 12).
- There are two lines that are hardly readable and another two line that are unreadable.
- The resulting image will feature a black frame of 3 pixels, as the convolution is exclusively applied to valid positions for the kernel.
- Half the size of the kernel will be removed from the image boundaries
- The results are nearly identical when using Gaussian filters to smooth the data.
- The main difference in the 1D filters is the small additional weight at both ends, which is almost zero, as shown in Fig 5 and Fig 6.

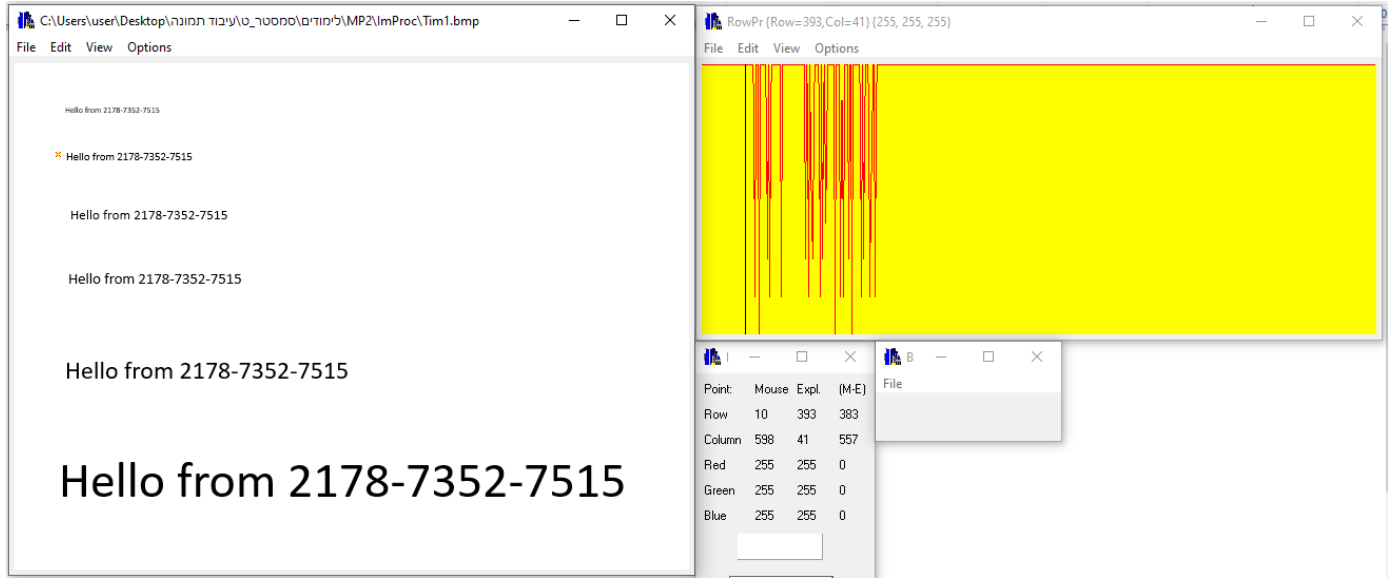


Fig 11: Profilers of Tim1.bmp



Fig 12: Profilers of Tim1LPF7c.bmp

MP2.13. Blur Image By Using FD Gaussian Filter and FFT

- FFT for Blurring:
  - The process begins by transforming the input image from the spatial domain to the frequency domain using FFT.
  - Gaussian filters are applied in the frequency domain to attenuate high-frequency components, resulting in a blurred image.
  - Convolution in the frequency domain is computationally efficient, reducing processing time compared to spatial domain operations.
- We designed a Gaussian filter (Fig 13) to blur the image (sigmaRadius = 80).
- The Gaussian filter in Frequency Domain (Fig 14): The Gaussian filter is a low-pass filter that attenuates high frequencies while preserving low frequencies.
- In the frequency domain, it's represented as a Gaussian-shaped function (Fig 15).

```
//Creates a 2D Gaussian filter and stores it in the given image.
void CreateGaussianFilter(c2dFloatGrayImage& image, tFloat sigmaRadius)
{
    tFloat x, y, r, z;
    double summa = 0;
    int numRows = image.NumberOfRows();
    int numColumns = image.NumberOfColumns();

    // Iterate over each pixel in the image
    for (int row = 0; row < numRows; row++)
    {
        for (int column = 0; column < numColumns; column++)
        {
            // Calculate the distances from the center of the image
            y = row - numRows / 2;
            x = column - numColumns / 2;
            // Calculate the radial distance
            r = sqrt(y * y + x * x);
            // Calculate the Gaussian function value based on the distance and sigma
            z = r / sigmaRadius;
            z = exp(-z * z);
            // Accumulate the sum of Gaussian values for normalization
            summa += z;
            // Store the Gaussian value in the image
            image.SetPixelValue(row, column, z);
        }
    }
}
```

Fig 13: Code for creating gaussian filter

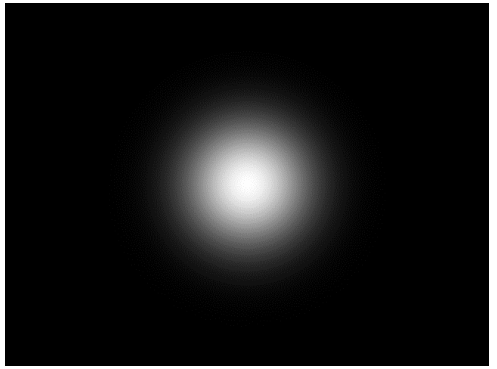


Fig 14: Gaussian Filter in FD

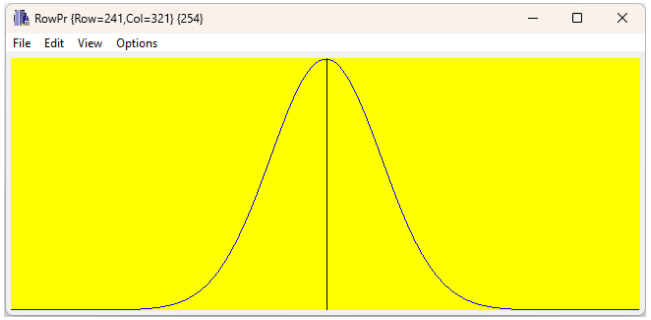


Fig 15: Profilers of Gaussian Filter in FD

MP2.13. Blur Image By Using FD Gaussian Filter and FFT

- We used the Gaussian filter in the frequency domain to blur an image by using ApplyFFT function (Fig 16).
- After creating the Gaussian filter, we are shifting the Fourier-transformed image by half of its size. This is necessary because:
  - After applying the FFT, low frequencies are located at the corners (top-left), while high frequencies are at the center.
  - Shifting ensures that the low-frequency components, which represent the overall structure of the image, are centered for easier manipulation.
- Then we transformed the Image to Frequency Domain: this is done using the Fast Fourier Transform (FFT).
- We applied the Gaussian Filter in Frequency Domain: Multiply the Fourier transformed image by the Gaussian filter in the frequency domain.
- At last, we transformed the filtered image back to the spatial domain using the inverse Fourier Transform.

```
// Applies Fast Fourier Transform (FFT) to the input image and filters it using a Gaussian filter.
c2dFloatGrayImage ApplyFFT(string inputFileName, bool reverseFilter, int radius)
{
    // Load the input image
    c2dFloatGrayImage imageRe(AddBMPExtention(inputFileName).c_str());
    c2dFloatGrayImage imageIm(imageRe.NumberOfRows(), imageRe.NumberOfColumns());
    // Create a Gaussian filter image with the same dimensions as the input image
    c2dFloatGrayImage gaussFilter(imageRe.NumberOfRows(), imageRe.NumberOfColumns());
    CreateGaussianFilter(gaussFilter, radius);
    if (reverseFilter)
    {
        CreateInverseGaussianFilter(gaussFilter, radius);
    }
    // Shift both real and imaginary part images
    ShiftHalfSize(imageRe);
    ShiftHalfSize(imageIm);
    // Perform Forward FFT on the images
    DoFFT(imageRe, imageIm, FORWARD_FFT, NORMALIZE_BY_SQRT);
    // Filter the image in the frequency domain
    DoFiltrationInFD(imageRe, imageIm, gaussFilter);
    // Perform Inverse FFT on the filtered images
    DoFFT(imageRe, imageIm, REVERSE_FFT, NORMALIZE_BY_SQRT);
    // Shift both real and imaginary part images
    ShiftHalfSize(imageRe);
    ShiftHalfSize(imageIm);
    return imageRe;
}
```

Fig 16: Code for applying FFT

MP2.13. Blur Image By Using FD Gaussian Filter and FFT

- We applied FD Gaussian filter over the original Tim1.bmp image (Fig 17) by using convolution.
- Applying a Gaussian low-pass filter (LPF) in the frequency domain (FD) typically results in attenuating higher frequency components of the signal while preserving the lower frequency components.
- This smoothing effect helps reduce noise and sharp transitions in the signal, resulting in a smoother and more regular frequency spectrum (Fig 18).
- There are two lines that are hardly readable and another two line that are unreadable.

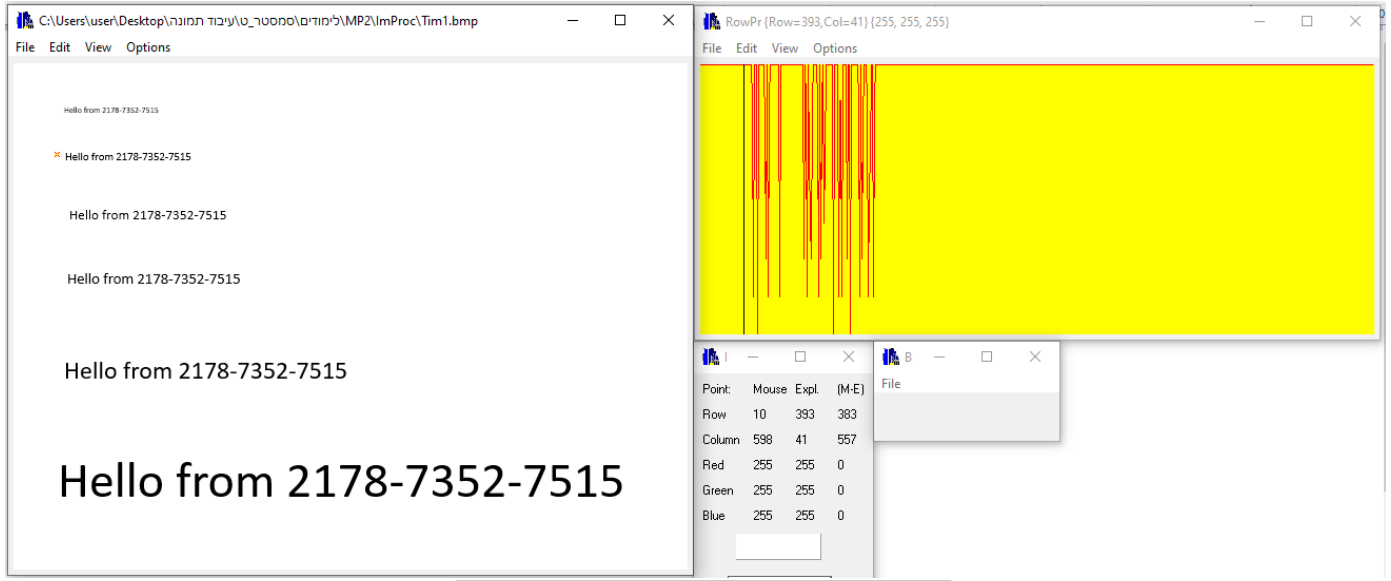


Fig 17: Profilers of Tim1.bmp



Fig 18: Profilers of Tim1LPFf.bmp

MP2.14. Restore Blurred Images – Convolution

- In our project, we aimed to sharpen blurry images using High Pass Filter (HPF) convolution.
- Sharpening enhances image details and edges, making them more prominent.
- It involves convolving the image with a sharpening kernel, which emphasizes high-contrast areas.
- Our approach involved testing various HPF filters, including Filters 1 through 7 (Fig 19) and others.
- After thorough experimentation, we opted to utilize Filter 6 and Filter 7 to attempt the restoration of the blurred figures.
  To distinguish the resulting images, we labeled them with the prefixes 'HPF1c' and 'HPF2c' respectively, indicating the specific filters applied.

Filter 1

Factor = 1/25

-1	-1	-1	-1	-1
-1	-1	-1	-1	-1
-1	-1	47	-1	-1
-1	-1	-1	-1	-1
-1	-1	-1	-1	-1

Filter 2

Factor = 1/256

-1	-4	-6	-4	-1
-4	-16	-24	-16	-4
-6	-24	476	-24	-6
-4	-16	-24	-16	-4
-1	-4	-6	-4	-1

Filter 3

Factor = 1

0	-1	0
-1	5	-1
0	-1	0

Filter 4

Factor = 1

-1	-1	-1
-1	9	-1
-1	-1	-1

Filter 5

Factor = 1/16

-1	-2	-1
-2	28	-2
-1	-2	-1

Filter 6

Factor = 1

-2	-2	-2
-2	17	-2
-2	-2	-2

Filter 7

Factor = 1/16

-8	-16	-8
-16	112	-16
-8	-16	-8

Fig 19: Various filters were tested

MP2.14. Restore Blurred Images – Convolution – HPF1c

- Filter HPF1c operates as a high-pass filter, achieved by subtracting the identity kernel and a low-pass filter.

HPF1c Design:

$$9*\left(\begin{bmatrix}0&0&0\\0&1&0\\0&0&0\end{bmatrix}-\frac{1}{9}*\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}\right)=\begin{bmatrix}-1&-1&-1\\-1&8&-1\\-1&-1&-1\end{bmatrix}\Rightarrow$$

$$\Rightarrow\begin{bmatrix}-1&-1&-1\\-1&8&-1\\-1&-1&-1\end{bmatrix}+\begin{bmatrix}0&0&0\\0&1&0\\0&0&0\end{bmatrix}=\begin{bmatrix}-1&-1&-1\\-1&9&-1\\-1&-1&-1\end{bmatrix}\Rightarrow\begin{bmatrix}-2&-2&-2\\-2&17&-2\\-2&-2&-2\end{bmatrix}$$

- To sharpen a filter, we begin by subtracting the identity kernel from the low-pass filter, transforming it into a high-pass filter.
- Following this, we modify the kernel by increasing the weight of the central pixel and decreasing the weights of the surrounding pixels.
- This adjustment aims to emphasize edges and details in the image.
- It's important to ensure that the sum of all weights in the kernel remains at 1 to preserve the overall average pixel intensity and avoid unintended alterations.

MP2.14. Restore Blurred Images – Convolution – HPF2c

- Filter HPF2c was achieved by subtracting a binomial filter from the kernel identity filter.

HPF2c Design:

$$\begin{bmatrix}0 & 0 & 0\\0 & 1 & 0\\0 & 0 & 0\end{bmatrix} - \frac{1}{16} * \begin{bmatrix}1 & 2 & 1\\2 & 4 & 2\\1 & 2 & 1\end{bmatrix} = \frac{1}{16} * \begin{bmatrix}-1 & -2 & -1\\-2 & 12 & -2\\-1 & -2 & -1\end{bmatrix} \Rightarrow$$

$$\Rightarrow \frac{1}{16} * \begin{bmatrix}-1 & -2 & -1\\-2 & 12 & -2\\-1 & -2 & -1\end{bmatrix} + \begin{bmatrix}0 & 0 & 0\\0 & 1 & 0\\0 & 0 & 0\end{bmatrix} = \frac{1}{16} * \begin{bmatrix}-1 & -2 & -1\\-2 & 28 & -2\\-1 & -2 & -1\end{bmatrix} \Rightarrow \frac{1}{16} * \begin{bmatrix}-8 & -16 & -8\\-16 & 112 & -16\\-8 & -16 & -8\end{bmatrix}$$

- The central value (112) is significantly larger than the surrounding values, indicating emphasis on the central pixel.
- The negative values surrounding the central pixel contribute to enhancing the contrast between adjacent pixels, enhancing the sharpening effect.
- The factor = 1/16 indicates that the filter values are normalized to ensure that the filter doesn't change the overall brightness of the image during convolution.
- Subtracting a binomial filter from the identity filter emphasizes high-frequency components in the image while suppressing low-frequency components, resulting in a sharpening effect.
- Overall, this filter enhances edges and details in an image, making them appear more pronounced, and it achieves this by emphasizing the central pixel while suppressing its neighboring pixels.

MP2.14. Restore Blurred Images – Convolution

- The function “ApplyHPFToBlurredImages” (Fig 20) applies high-pass filters (HPF) to a set of blurred images.
- Two different high-pass filter kernels, HPF1 and HPF2, are defined.
- The function iterates over a vector of input file names.
- For each input image:
  - It loads the image.
  - It initializes a destination image with the same dimensions as the source image.
  - It applies convolution filtering using the DoFiltrationByConvolution function twice.
  - Once with HPF1 and a factor of 1.
  - Again, with HPF2 and a factor of HTP2factor, which is set to 1.0 / 16.0.
  - The filtered images are saved with filenames constructed by appending prefixes to the original input filenames.

```
//Applies High Pass Filters (HPF) to the blurred images specified by the input filenames.
void ApplyHPFToBlurredImages(vector<string> inputFileNames)
{
    // Define High Pass Filter kernels
    double HPF1[3][3] = { {-2, -2, -2}, {-2, 17, -2}, {-2, -2, -2} };
    double HPF2[3][3] = { {-8, -16 , -8}, {-16, 112 , -16}, {-8, -16 , -8} };
    // Define normalization factor for HPF2
    double HTP2factor = 1.0 / 16.0;
    int fileNamesCount = inputFileNames.size();
    for (int i = 0; i < fileNamesCount; i++)
    {
        // Load the input image
        c2dByteGrayImage source(AddBMPExtention(inputFileNames[i]).c_str());
        c2dByteGrayImage dest(source.NumberOfRows(), source.NumberOfColumns());
        // Apply HPF1 to the source image
        DoFiltrationByConvolution(source, dest, HPF1, 1);
        dest.SaveToGrayBmpFile(AddBMPExtention("HPF1c" + inputFileNames[i]).c_str());
        // Apply HPF2 to the source image
        DoFiltrationByConvolution(source, dest, HPF2, HTP2factor);
        dest.SaveToGrayBmpFile(AddBMPExtention("HPF2c" + inputFileNames[i]).c_str());
    }
}
```

Fig 20: Code for ApplyHPFToBlurredImages function

MP2.14. Restore Blurred Images – Convolution

- The function “DoFiltrationByConvolution” (Fig 21) performs image filtration using convolution with a given filter kernel.
- It takes the source image, the destination image, the filter kernel, and a normalization factor as input parameters.
- It initializes the destination image by clearing it.
- It iterates over the rows and columns of the source image, excluding the border pixels according to the size of the filter kernel.
- For each pixel in the source image, It computes the convolution sum by multiplying each pixel value in the local neighborhood by the corresponding filter coefficient and summing them up.
- The result is multiplied by the normalization factor.
- The resulting value is clamped between 0 and 255.
- The filtered pixel value is stored in the corresponding position in the destination image.
- The function modifies the destination image in place, performing the filtration operation.

```
// Applies a convolution filter to the source image and stores the result in the destination image.
void DoFiltrationByConvolution(c2dByteGrayImage& source,
    c2dByteGrayImage& dest, double filter[][3], double factor)
{
    int filterHalfSize = 1;
    // Clear the destination image
    dest.Clear();
    int numRows = source.NumberOfRows();
    int numColumns = source.NumberOfColumns();
    for (int row = filterHalfSize; row < numRows - filterHalfSize; row++)
    {
        for (int column = filterHalfSize; column < numColumns - filterHalfSize; column++)
        {
            double summa = 0;
            for (int y = -filterHalfSize; y <= filterHalfSize; y++)
            {
                for (int x = -filterHalfSize; x <= filterHalfSize; x++)
                {
                    // Perform the convolution operation
                    summa += filter[filterHalfSize + y][filterHalfSize + x] *
                        (double)source.GetPixelValue(row + y, column + x);
                } // of x
            } // of y
            summa = summa * factor;
            // Clamp the result to the range [0, 255]
            if (summa < 0) summa = 0;
            if (summa > 255) summa = 255;
            dest.SetPixelValue(row, column, (tByte)(summa + 0.5));
        } // of column
    } // of row
}
```

Fig 21: Code for DoFiltrationByConvolution function

MP2.15. Restore Blurred Images – FFT

- We created a high-pass filter (HPF –Fig 22) Inversed Gaussian filter in the frequency domain (FD- Fig 23) with sigmaRadius = 80.
- This filter acts as the inverse of the original Gaussian LPF in the frequency domain as described in slide MP2.4.
- Our aim with this filter was to effectively restore the Gaussian filter in the frequency domain (FD). Multiplying the filter and its inverse in FD results in the identity filter, achieving nearly perfect restoration.
- In the frequency domain, it's represented as a Gaussian-shaped function (Fig 24).
- For each pixel, It then calculates the total squared distance from the center and divides it by the square of the radius. This operation effectively scales the distance by the standard deviation.
- Next, it computes the inverse Gaussian: 1 - exp(-total) and scales it by 255.0. This is a scaling factor to convert the result to the range of 0 to 255, which is typical for grayscale pixel values.
- Then, we use the Inversed Gaussian filter we created and function ApplyFFT to apply Filtration in FD (as described before in figure 16 and slide MP2.4).

```
// Creates an inverse Gaussian filter and stores it in the given image.
void CreateInverseGaussianFilter(c2dFloatGrayImage& image, tFloat sigmaRadius)
{
    tFloat x, y, z, total;
    double summa = 0;
    int numRows = image.NumberOfRows();
    int numColumns = image.NumberOfColumns();
    for (int row = 0; row < numRows; row++)
    {
        for (int column = 0; column < numColumns; column++)
        {
            // Calculate squared distances from the center of the image
            y = (row - numRows / 2) * (row - numRows / 2);
            x = (column - numColumns / 2) * (column - numColumns / 2);
            // Calculate the total squared distance divided by sigma squared
            total = (x + y) / (sigmaRadius * sigmaRadius);
            // Apply the inverse Gaussian function and scaling
            z = 255.0 * (1 - exp(-total));
            image.SetPixelValue(row, column, z);
        }
    }
}
```

Fig 22: Code for inverse gaussian filter

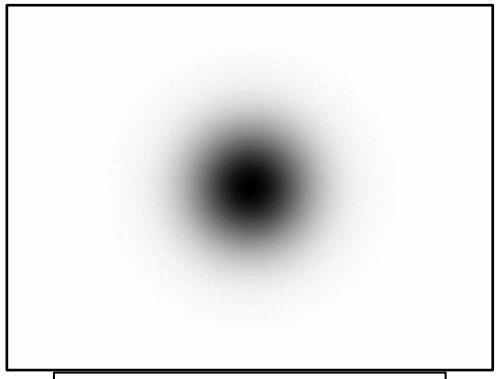


Fig 23: Gaussian Filter in FD

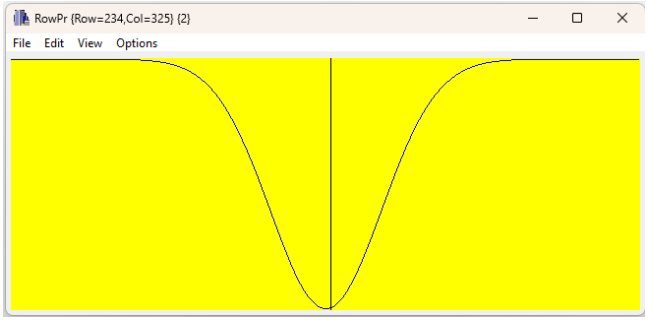


Fig 24: Profilers of Gaussian Filter in FD

MP2.16. Restore Blurred Images – Results

- We utilized the set of three High Pass Filters (HPFs) that we designed specifically for this task to enhance the clarity and detail of the three blurred images.
- Taking a systematic approach, we systematically applied each HPF to every individual blurred image in the set.
- Afterward, we carefully analyzed the effects and improvements brought about by each HPF on the respective blurred image it was applied to.
- This comprehensive evaluation allowed us to gain insights into the efficacy and performance of each HPF in terms of enhancing image sharpness and detail.

MP2.16. Restore Blurred Images – Results – HPF1c

- We utilized HPF1c on the blurred image Tim1LPF5c.bmp (Fig 25).
- The result is shown in Figure 26.
- Looking at Figure 26, we notice faster and more noticeable changes in the values.
- The sharpened image has a wider range of pixel values, particularly in areas with sharp transitions, edges, and fine details.
- In the blurry image, the second smallest text is unreadable. But in the sharpened image, it becomes readable.
- Unfortunately, the smallest text is still unreadable in the sharpened version.

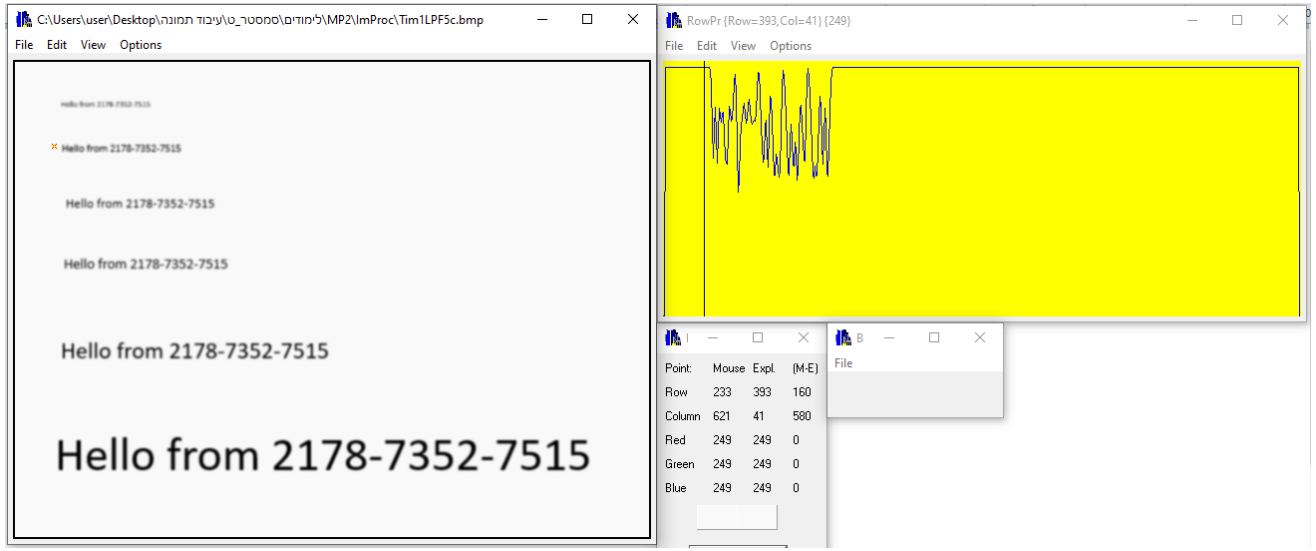


Fig 25: Profilers for Tim1LPF5c.bmp

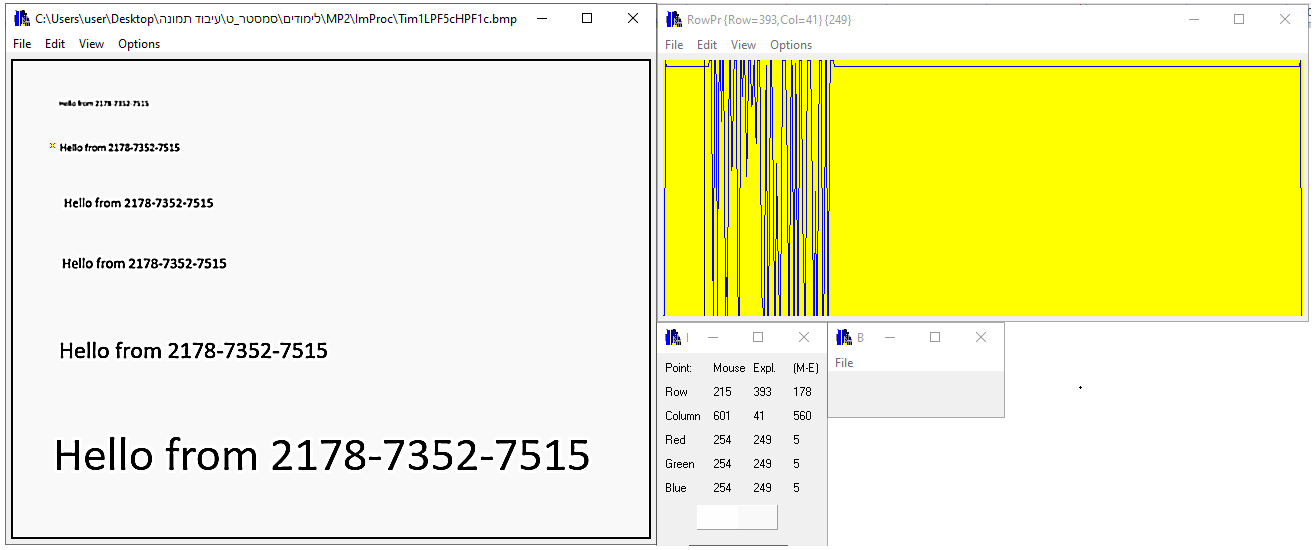


Fig 26: Profilers for HPF1cTim1LPF5c.bmp

MP2.16. Restore Blurred Images – Results – HPF1c

- We utilized HPF1c on the blurred image Tim1LPF7c.bmp (Fig 27).
- The result is shown in Figure 28.
- Looking at Figure 28, we notice faster and more noticeable changes in the values.
- The sharpened image has a wider range of pixel values, particularly in areas with sharp transitions, edges, and fine details.
- In the blurry image, the second smallest text is unreadable. But in the sharpened image, it becomes readable.
- Unfortunately, the smallest text is still unreadable in the sharpened version.
- The results in Figures 26 and 28 are very similar because the only distinction is the size of the Low Pass Filters (LPFs), which are essentially the same except for their sizes.

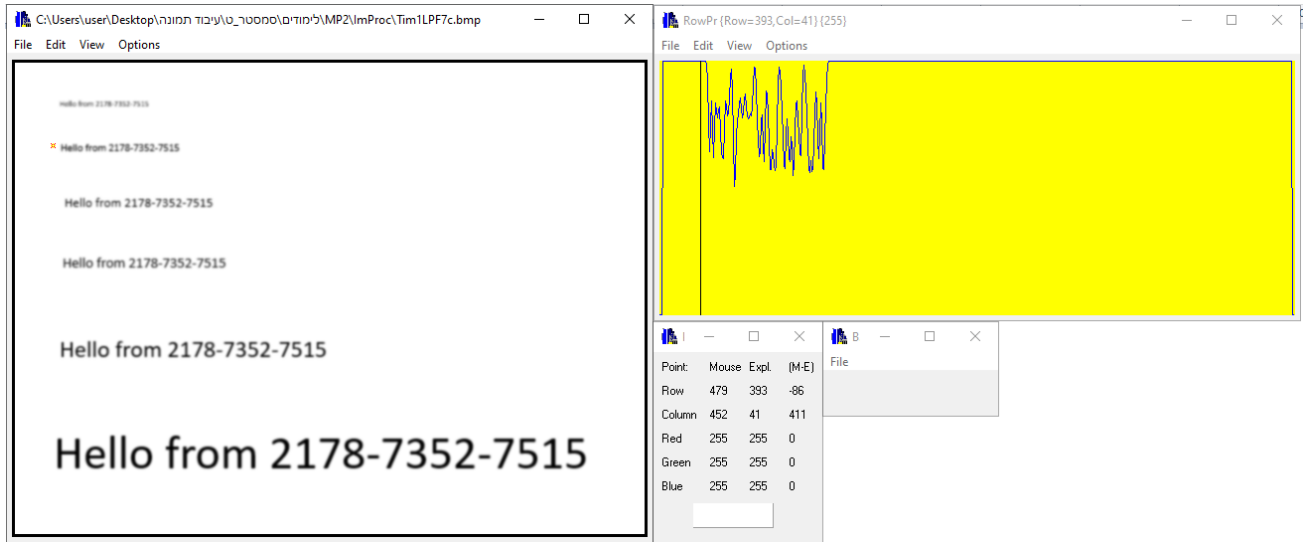


Fig 27: Profilers for Tim1LPF7c.bmp

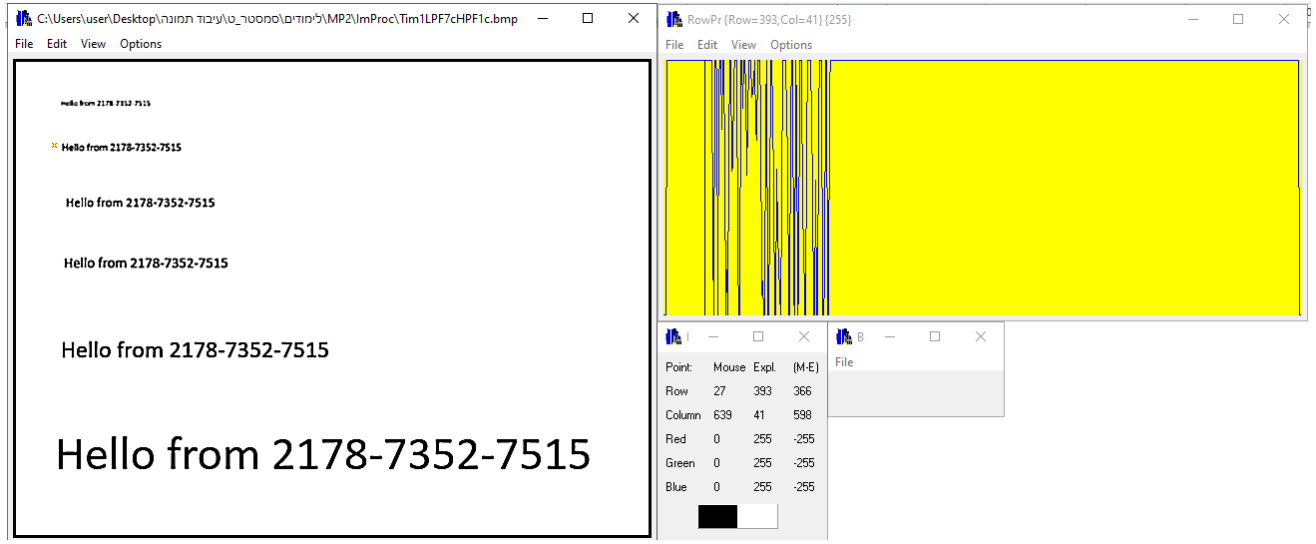


Fig 28: Profilers for HPF1cTim1LPF7c.bmp

MP2.16. Restore Blurred Images – Results – HPF1c

- We utilized HPF1c on the blurred image Tim1LPFf.bmp (Fig 29).
- The result is shown in Figure 30.
- Looking at Figure 30, we notice that the image has a wider range of pixel values, particularly in areas with sharp transitions, edges, and fine details.
- In the blurry image, the third smallest text is unreadable. But in the sharpened image, it becomes readable.
- Nonetheless, even after sharpening, the smallest two smallest text lines remains unreadable.



Fig 29: Profilers for Tim1LPFf.bmp

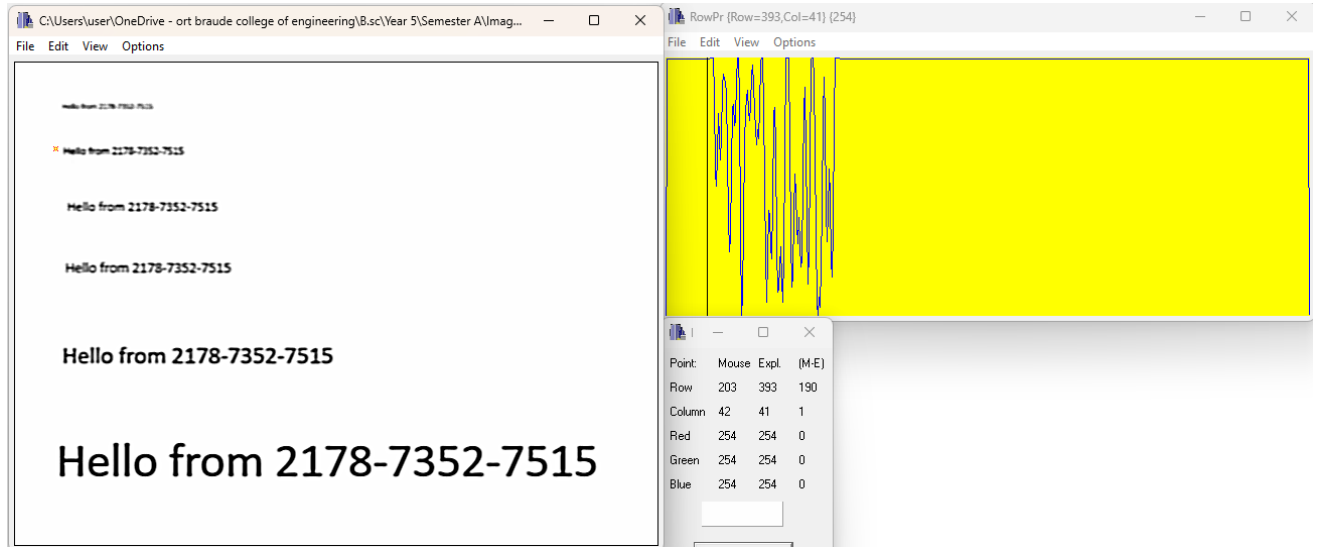


Fig 30: Profilers for HPF1cTim1LPFf.bmp

MP2.16. Restore Blurred Images – Results – HPF2c

- We utilized HPF2c on the blurred image Tim1LPF5c.bmp (Fig 31).
- The result is shown in Figure 32.
- Looking at Figure 32, we notice faster and more noticeable changes in the values.
- The sharpened image has a wider range of pixel values, particularly in areas with sharp transitions, edges, and fine details.
- In the blurry image, the second smallest text is unreadable. But in the sharpened image, it becomes readable.
- Unfortunately, the smallest text is still unreadable in the sharpened version.

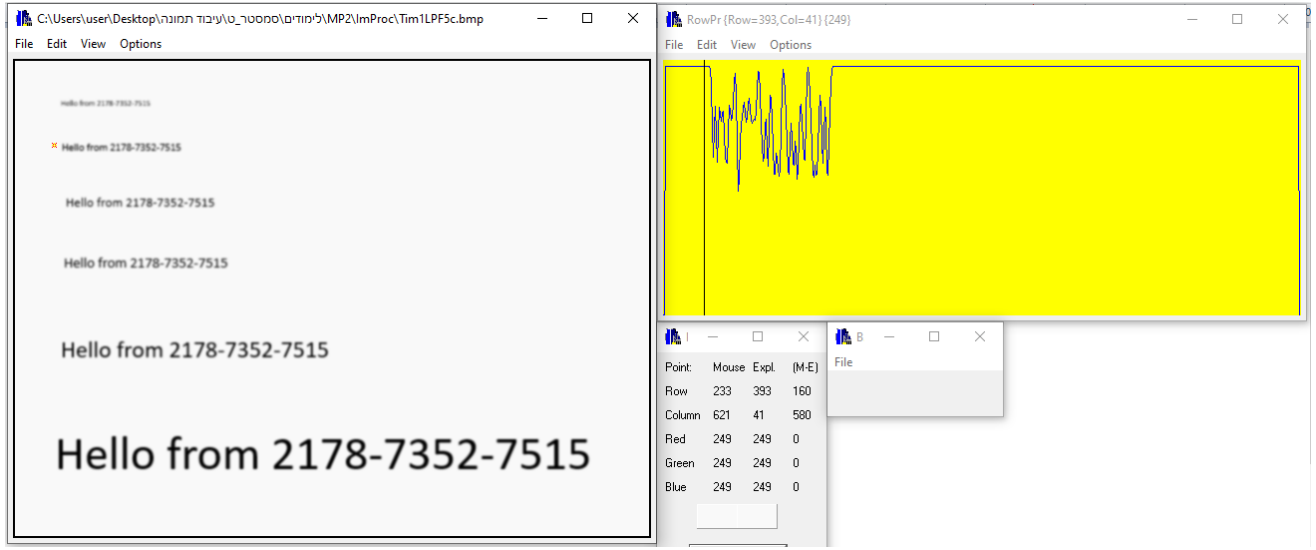


Fig 31: Profilers for Tim1LPF5c.bmp

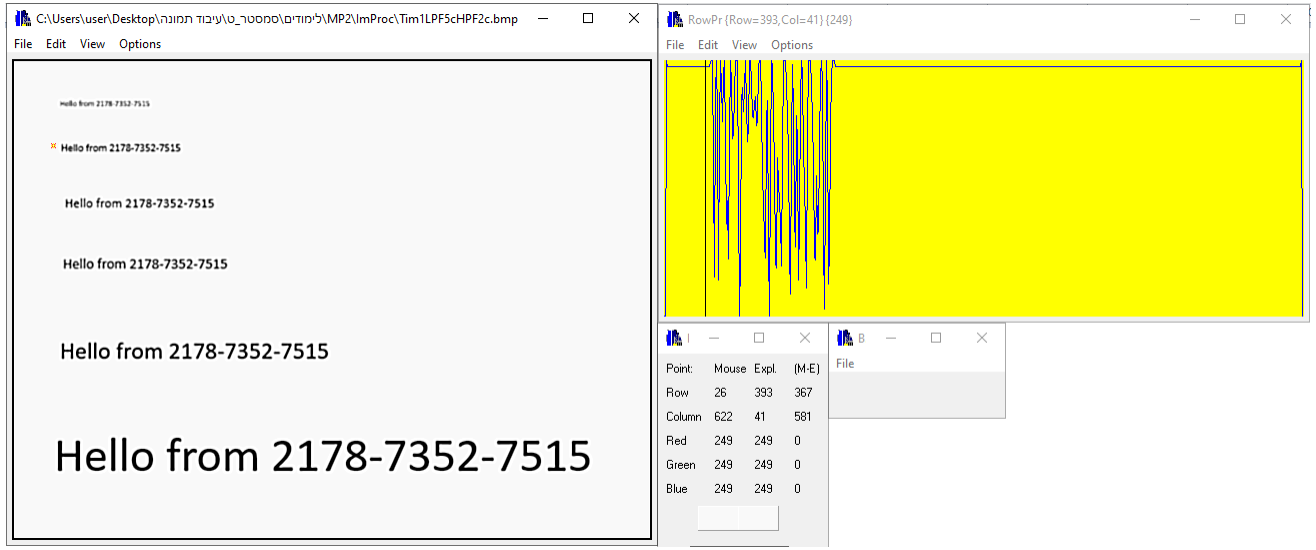


Fig 32: Profilers for HPF2cTim1LPF5c.bmp

MP2.16. Restore Blurred Images – Results – HPF2c

- We utilized HPF2c on the blurred image Tim1LPF7c.bmp (Fig 33).
- The result is shown in Figure 34.
- Looking at Figure 34, the sharpened image has a wider range of pixel values.
- In the blurry image, the second smallest text is unreadable. But in the sharpened image, it becomes readable.
- Nonetheless, even after sharpening, the smallest font remains unreadable.
- The results in Figures 32 and 34 are very similar because the only distinction is the size of the Low Pass Filters (LPFs), which are essentially the same except for their sizes.

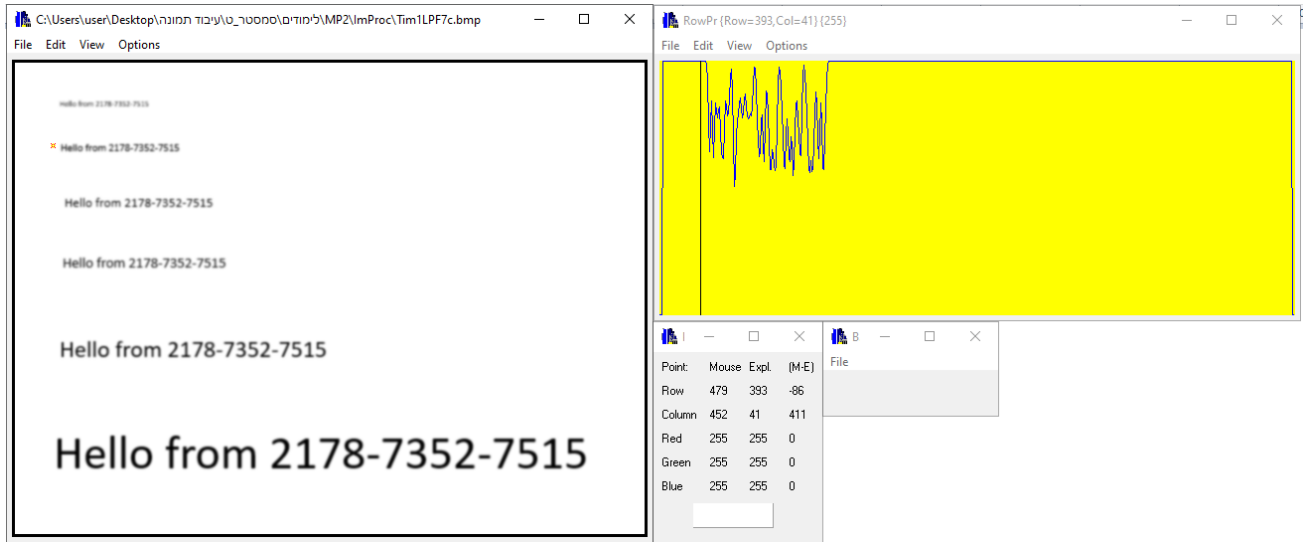


Fig 33: Profilers for Tim1LPF7c.bmp

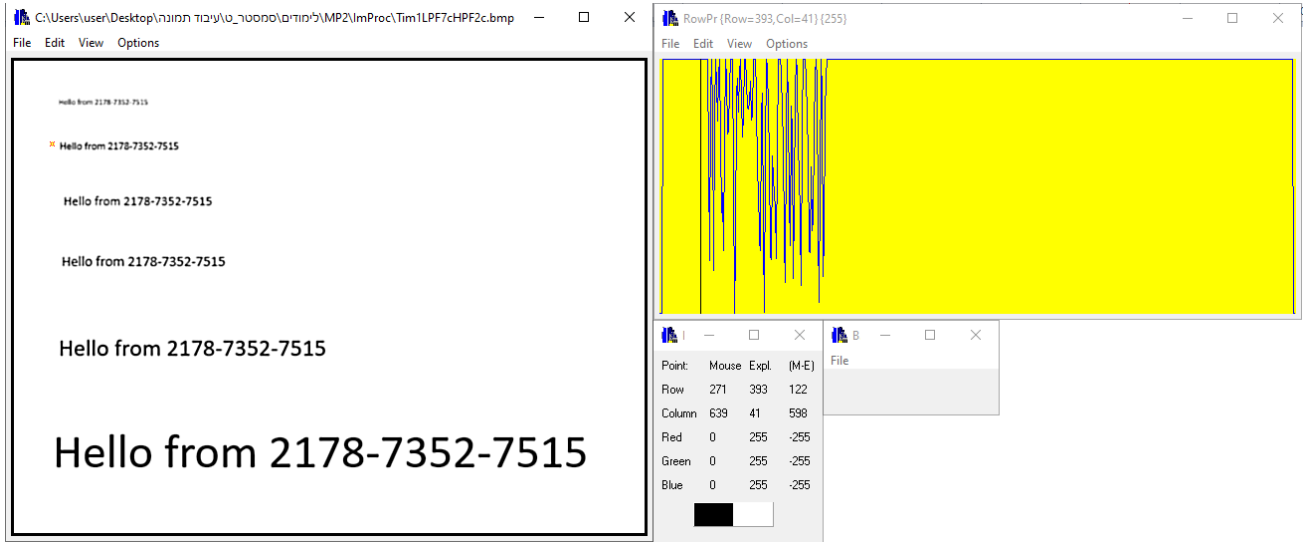


Fig 34: Profilers for HPF2cTim1LPF7c.bmp

MP2.16. Restore Blurred Images – Results – HPF2c

- We utilized HPF2c on the blurred image Tim1LPFf.bmp (Fig 35).
- The result is shown in Figure 36.
- Looking at Figure 36, we notice a wider range of pixel values.
- In the blurry image, the third smallest text is unreadable. But in the sharpened image, it becomes barely readable.
- The two smallest text lines are still unreadable in the sharpened version.

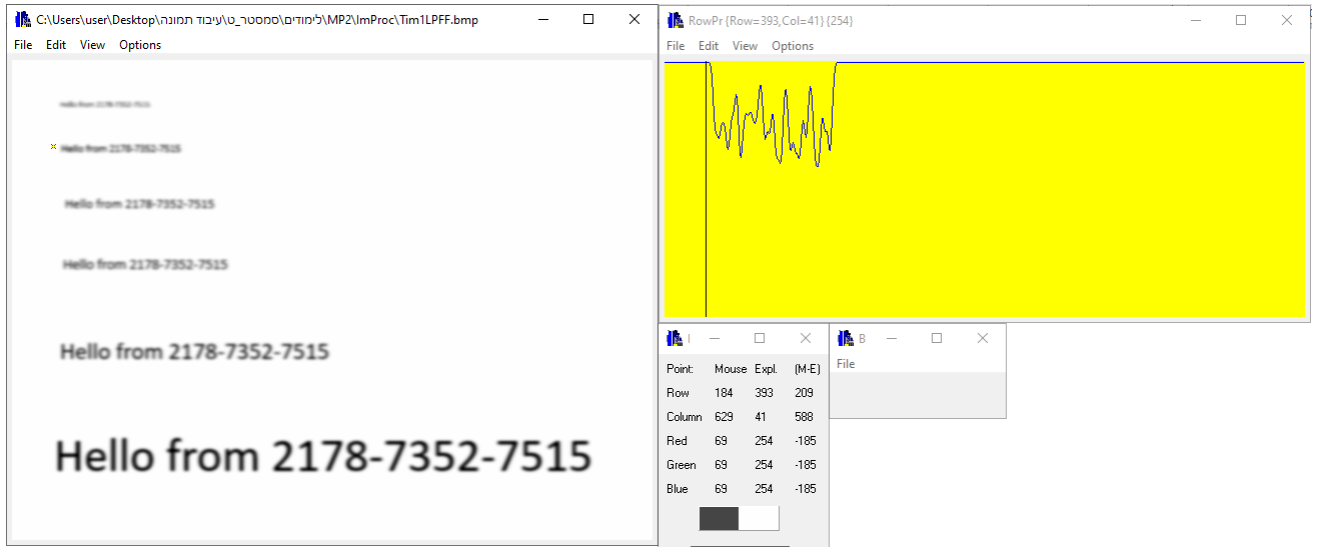


Fig 35: Profilers for Tim1LPFf.bmp

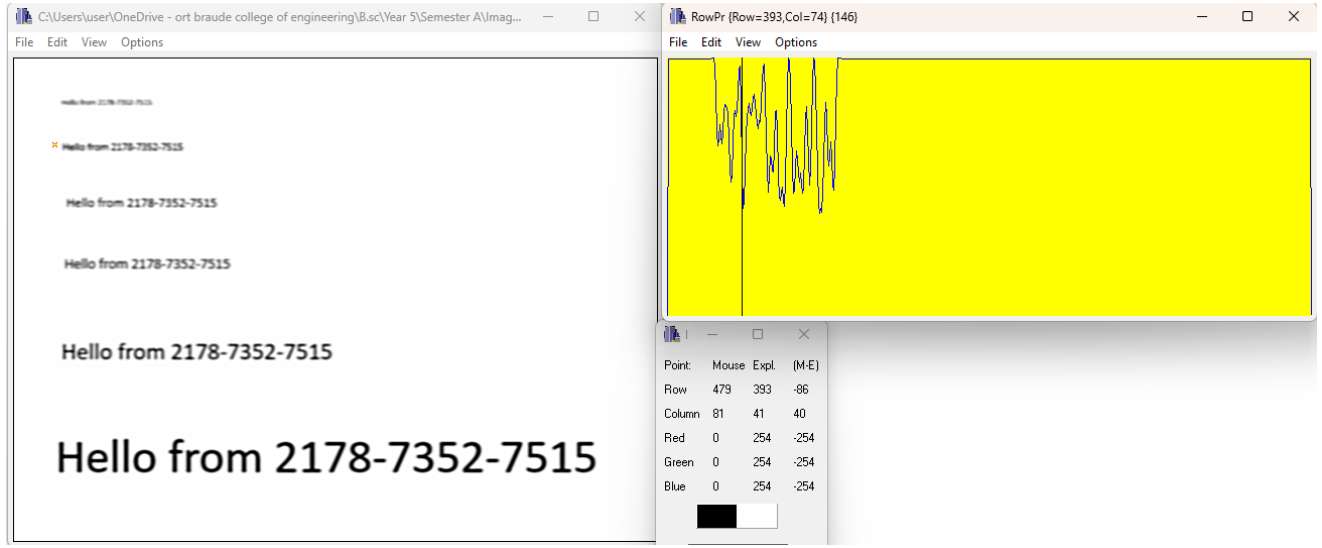


Fig 36: Profilers for HPF2cTim1LPFf.bmp

MP2.16. Restore Blurred Images – Results – HPFf

- We utilized HPFf (in the frequency domain) on the blurred image Tim1LPF5c.bmp (Fig 37).
- The result is shown in Figure 38.
- Looking at Figure 38, there are noticeable and swift changes in values.
- The two smallest lines are unreadable in the blurry image. The smallest line remains unreadable in the sharpened image, but the second line is slightly readable in the resulted image.
- The pixel values along the figure's border change due to the HPF's edge detection.
- The HPF detects edges because the border contrasts black against a white background.

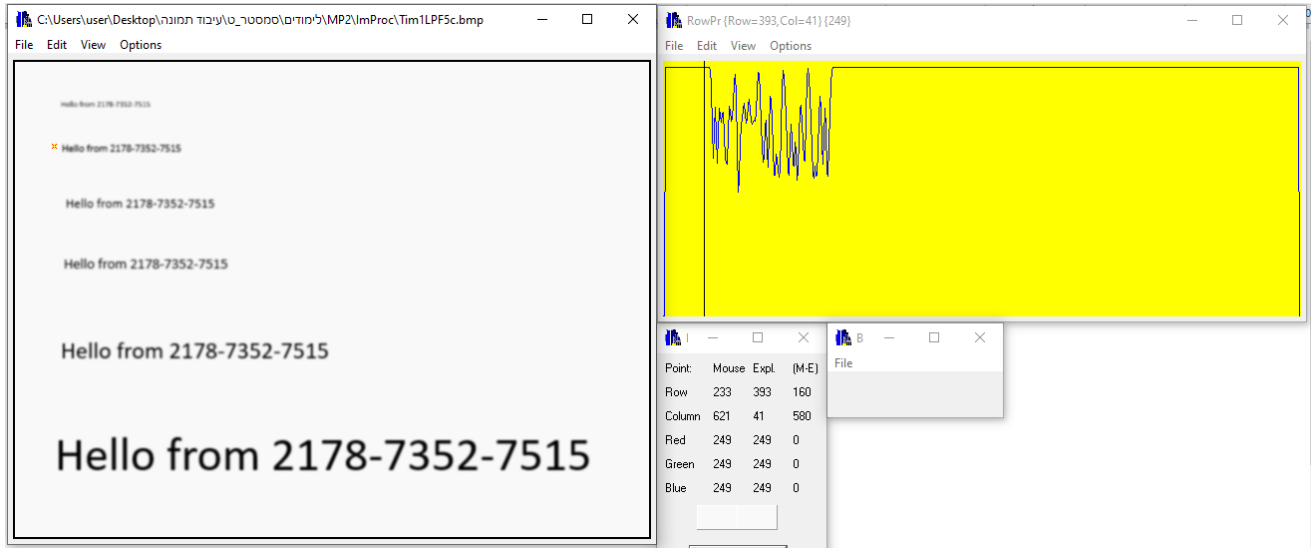


Fig 37: Profilers for Tim1LPF5c.bmp

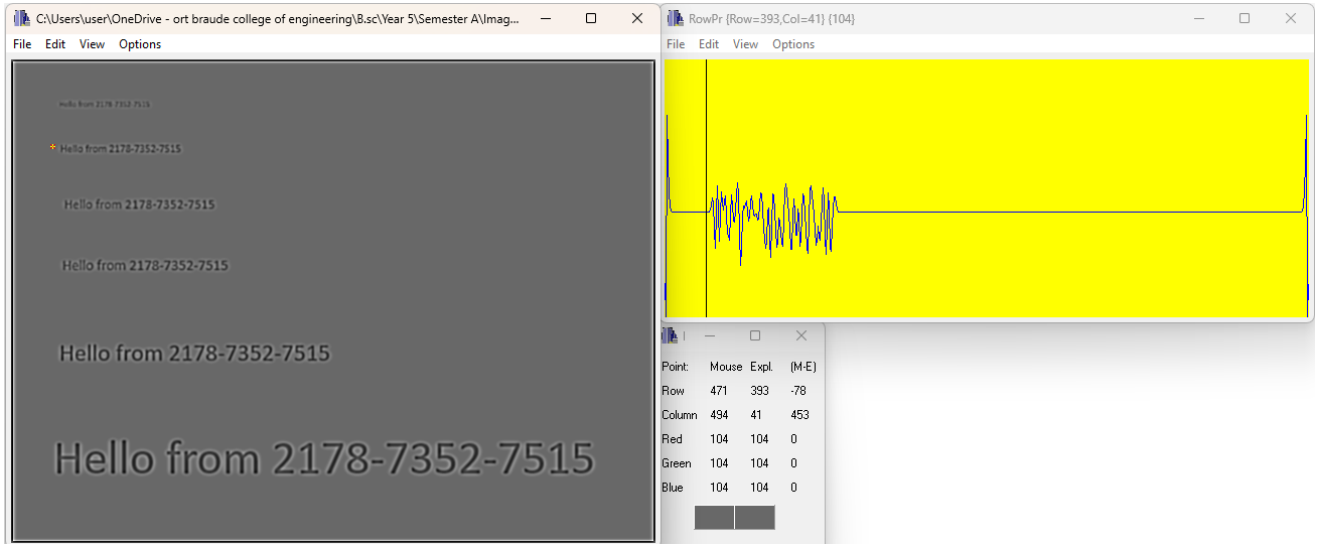


Fig 38: Profilers for HPFfTim1LPF5c.bmp

MP2.16. Restore Blurred Images – Results – HPFf

- We utilized HPFf on the blurred image Tim1LPF7c.bmp (Fig 39).
- The result is shown in Figure 40.
- Looking at Figure 40, there are noticeable and swift changes in values.
- The two smallest lines are unreadable in the blurry image. The smallest line remains unreadable in the sharpened image, but the second line is slightly readable in the resulted image.
- The pixel values along the figure's border change due to the HPF's edge detection.
- The HPF detects edges because the border contrasts black against a white background
- The results in Figures 38 and 40 are very similar because the only distinction is the size of the Low Pass Filters (LPFs), which are essentially the same except for their sizes.

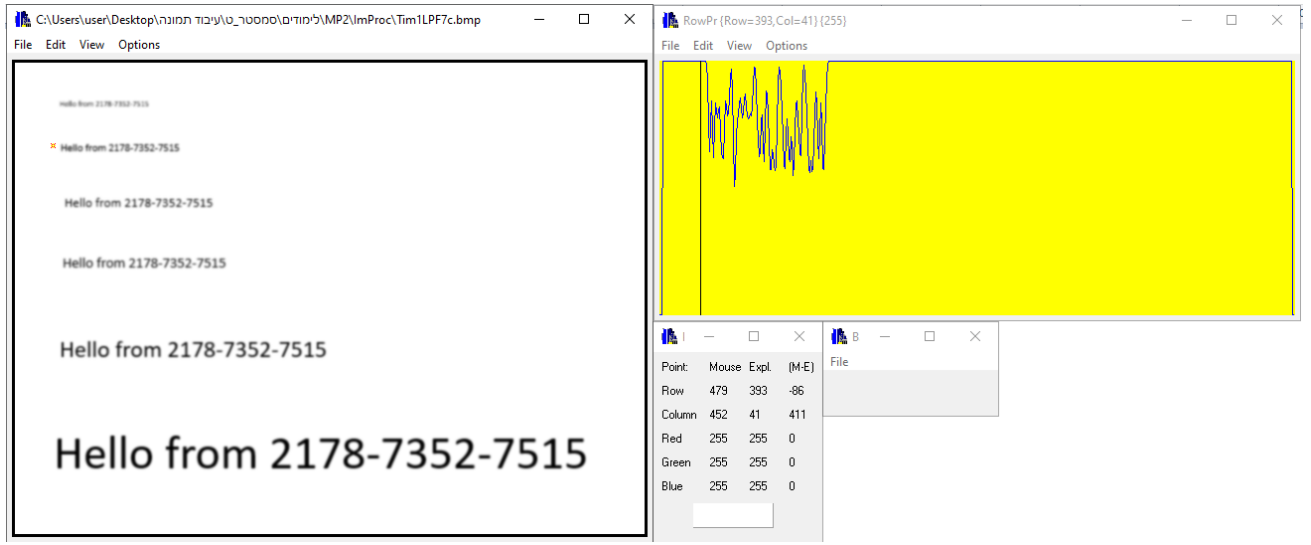


Fig 39: Profilers for Tim1LPF7c.bmp

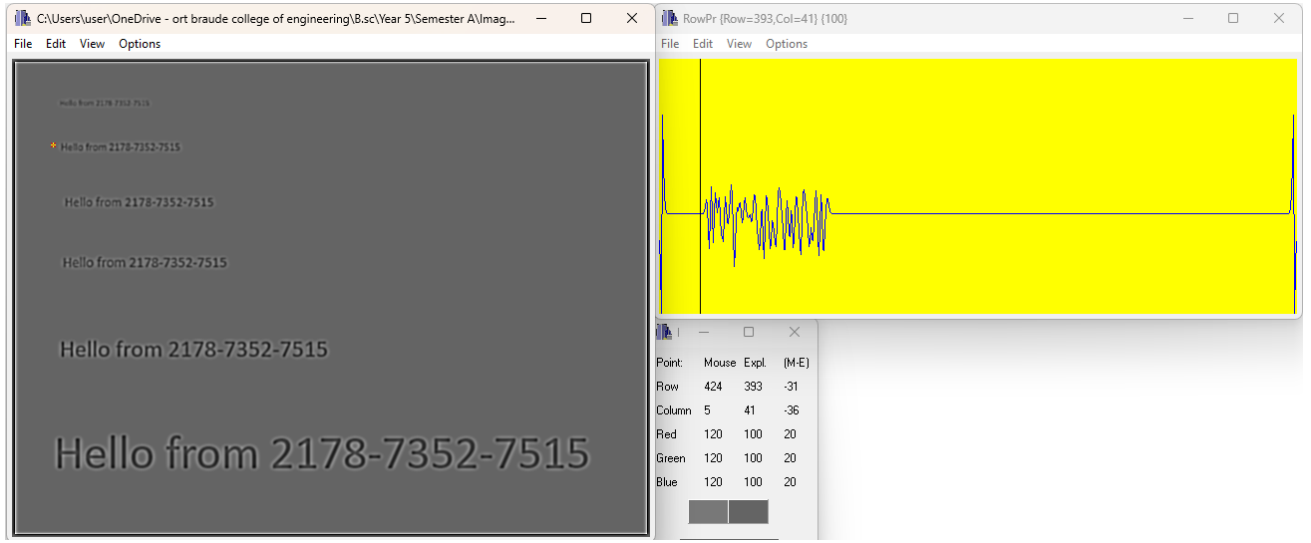


Fig 40: Profilers for HPFfTim1LPF7c.bmp

MP2.16. Restore Blurred Images – Results – HPFf

- We utilized HPFf on the blurred image Tim1LPFf.bmp (Fig 41). the result is shown in Figure 42.
- Inspection in the row viewer of Figure 42 reveals more significant and rapid changes in the values.
- The high-pass filter (HPF) in the frequency domain (FD) is applied after the low-pass filter (LPF) in the FD. These filters act as inverses, aiming for nearly perfect restoration.
- The HPF detects edges because the border contrasts black against a white background
- The darker appearance of the background in the resulting image after applying a Gaussian LPF is primarily due to the blurring effect, which reduces contrast and can lead to the loss of high-frequency information contributing to the background's brightness.



Fig 41: Profilers for Tim1LPFf.bmp

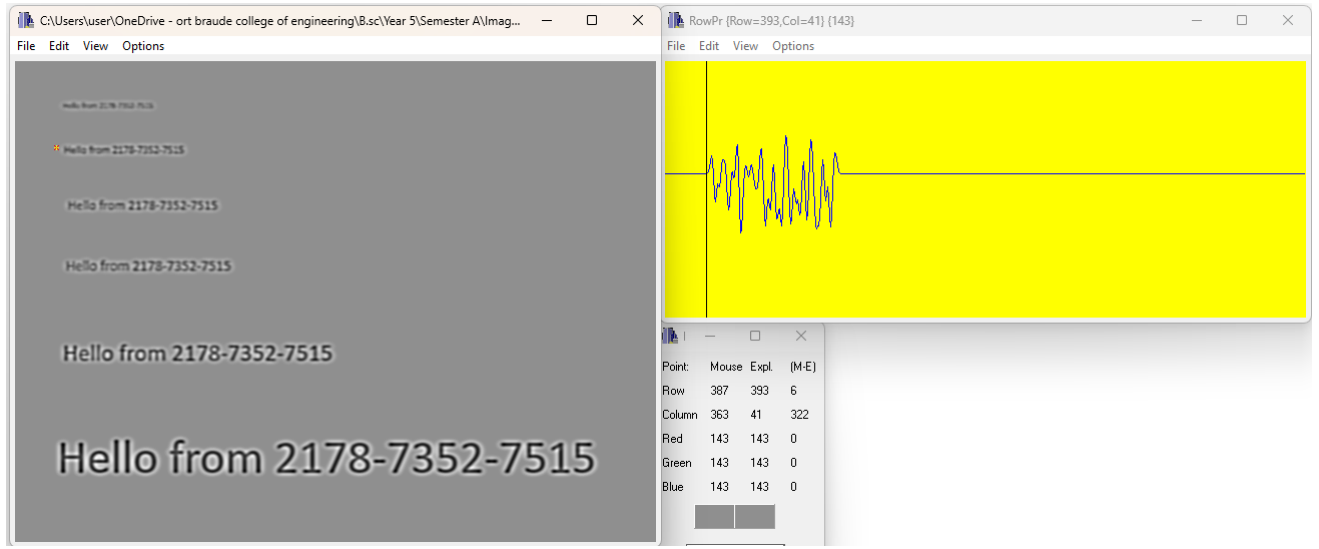
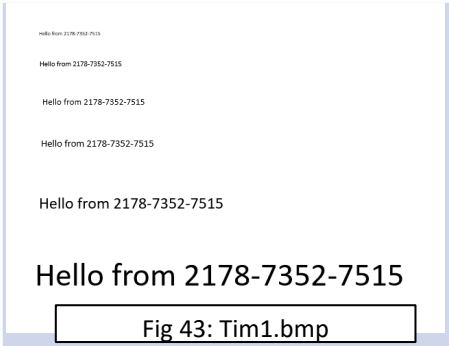
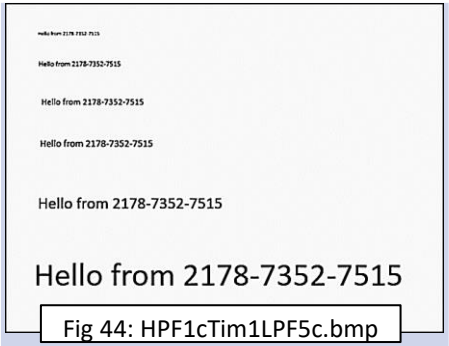
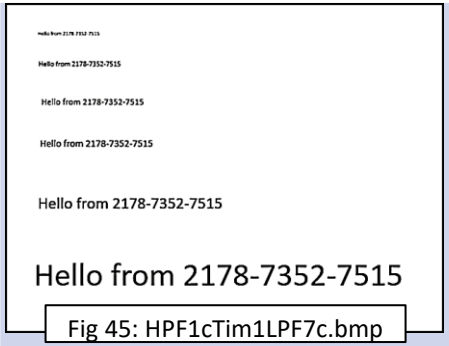
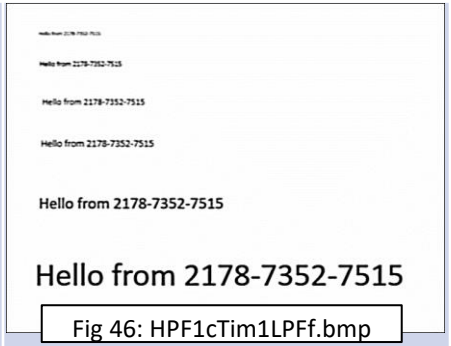
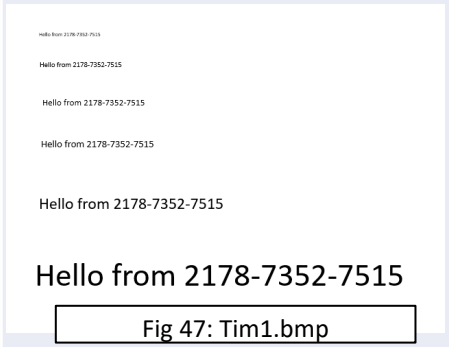
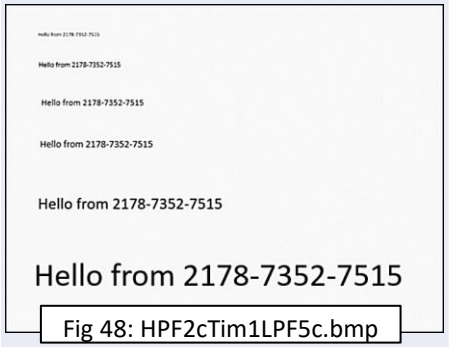
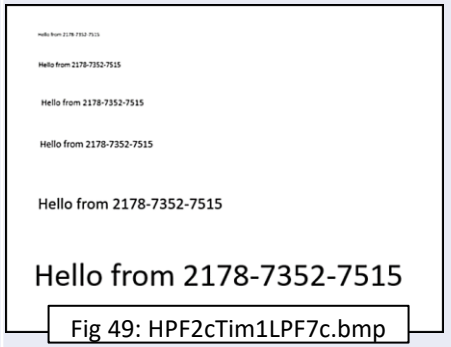
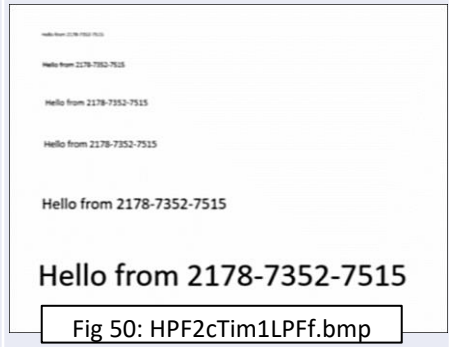
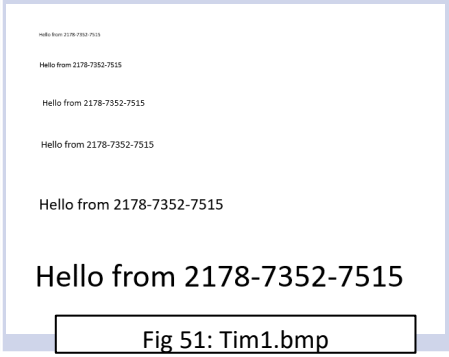
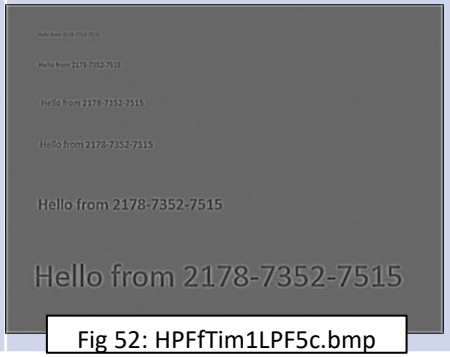
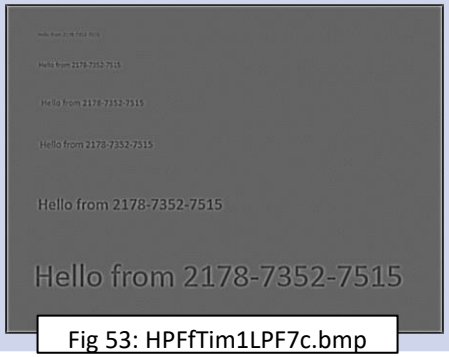
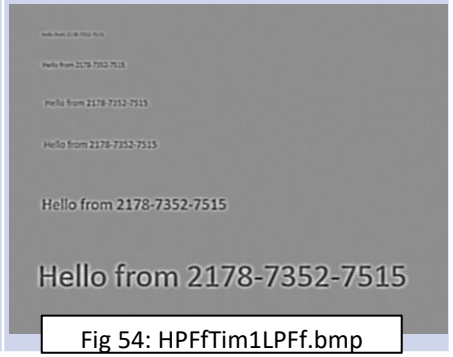


Fig 42: Profilers for HPFfTim1LPFf.bmp

MP2.16. Restore Blurred Images – Discussion

HPF Used	Original Image	Restored Blurred Image (After LPF5c)	Restored Blurred Image (After LPF7c)	Restored Blurred Image (After LPFf)
HPF1c	 Fig 43: Tim1.bmp	 Fig 44: HPF1cTim1LPF5c.bmp	 Fig 45: HPF1cTim1LPF7c.bmp	 Fig 46: HPF1cTim1LPFf.bmp
HPF2c	 Fig 47: Tim1.bmp	 Fig 48: HPF2cTim1LPF5c.bmp	 Fig 49: HPF2cTim1LPF7c.bmp	 Fig 50: HPF2cTim1LPFf.bmp
HPFf	 Fig 51: Tim1.bmp	 Fig 52: HPFfTim1LPF5c.bmp	 Fig 53: HPFfTim1LPF7c.bmp	 Fig 54: HPFfTim1LPFf.bmp

- Filter HPF2c produces a sharpened image, but with a smoother results compared to Filter HPF1c.
- The results of restored blurred images after LPF5c and LPF7c (Figs 44-45 for example) are very similar because the only distinction is the size of the Low Pass Filters (LPFs), which are essentially the same except for their sizes.
- The darker appearance of the background in the resulting image after applying a Gaussian LPF is primarily due to the blurring effect, which reduces contrast and can lead to the loss of high-frequency information contributing to the background's brightness.

MP2.21. TV Test BMP Image – Tim2.bmp

- Now that we've successfully completed the steps on the image Tim1.bmp, the next phase involves replicating these procedures on another image, specifically the grayImage13bmp from Assignment 13.
- We renamed grayImage13bmp as Tim2.bmp (Fig 55) to maintain consistency with our naming convention.
- The size of the image is 320x240 which is different the VGA size that we used in the previous phase.
- Let's proceed with the next steps to extend our findings and insights to a broader context.
- We use the exact same code as before.

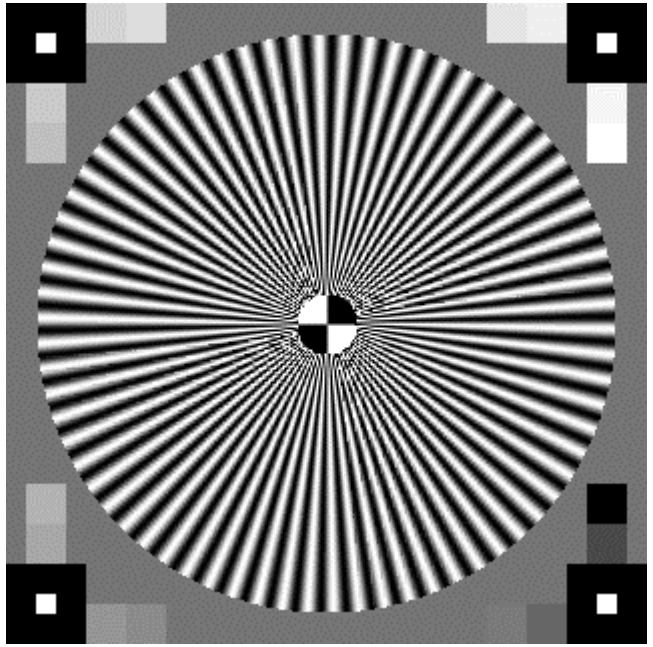


Fig 55: grayImage13bmp – Tim2.bmp

MP2.22. Blur Image By Using Fast Gaussian Filters and Convolution

- We utilized the same Fast Gaussian (1D) filters for blurring the images as we had employed during the initial stage of the project (Fig 56,57).

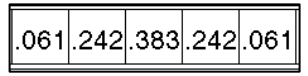


Fig 56: 1D Fast Gaussian Filter- Size 5

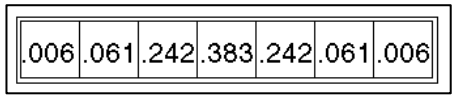


Fig 57: 1D Fast Gaussian Filter- Size 7

MP2.22. Blur Image By Using Fast Gaussian Filters and Convolution

- We applied Fast Gaussian filter (1D Filter) with size=5 over the original Tim2.bmp image (Fig 56) by using convolution.
- The original image has a central circular pattern with radiating black and white lines. The details in this image are sharp and clear.
- Applying the Fast Gaussian filter, will result in the image being smoothed or blurred (Fig 57).
- The filtered image shows a similar central circular pattern but with slightly blurred radiating lines due to the application of the LPF.
- The application of the LPF has resulted in a reduction of high-frequency content in the image, which is typically associated with sharp edges and fine details. This results in a blurrier or smoother appearance of the image. This is a common effect of LPF filtering in image processing.
- Half the size of the kernel will be removed from the image boundaries

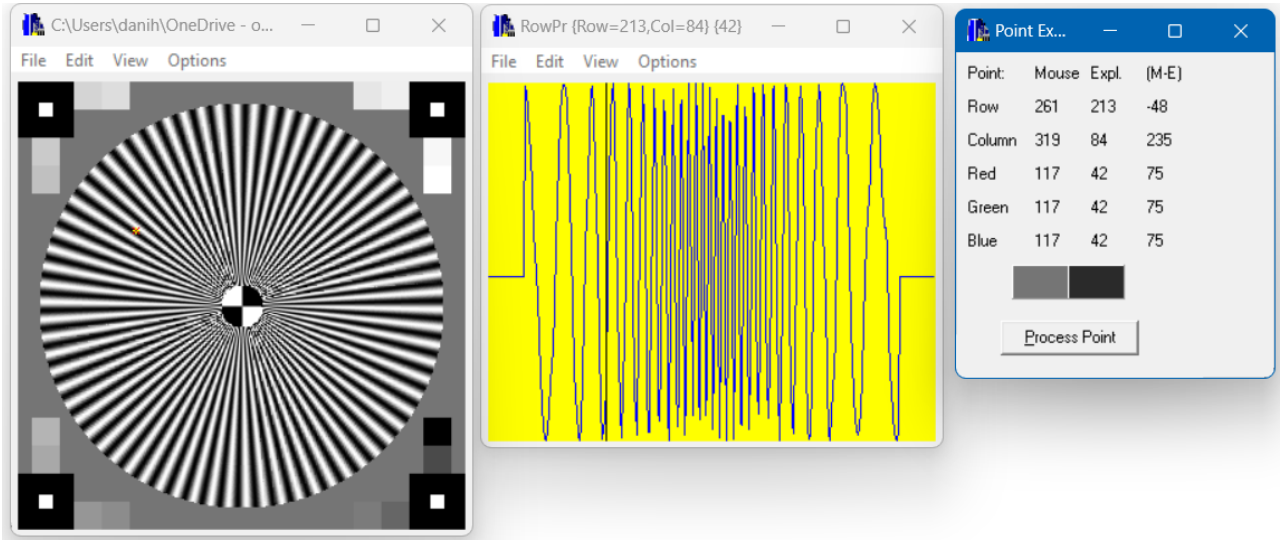


Fig 56: Profilers of Tim2.bmp

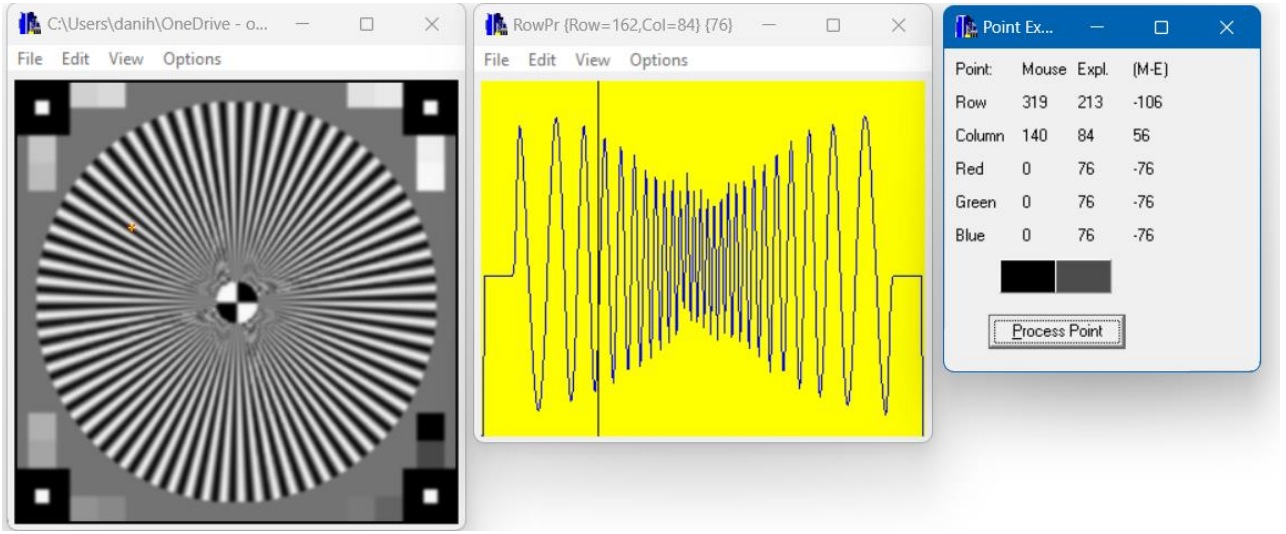


Fig 57: Profilers of Tim2LPF5c.bmp

MP2.22. Blur Image By Using Fast Gaussian Filters and Convolution

- We applied Fast Gaussian filter (1D Filter) with size=7 over the original Tim2.bmp image (Fig 58) by using convolution.
- Applying the Fast Gaussian filter, will result in the image being smoothed or blurred (Fig 59).
- The filtered image shows a similar central circular pattern but with slightly blurred radiating lines due to the application of the LPF as before.
- The application of the LPF has again resulted in a reduction of high-frequency content in the image, which is typically associated with sharp edges and fine details.
- Comparing the 2 blurred images (Fig 57, 59), the difference is barely noticeable, and the images appearing almost the same.
- The main difference in the 1D filters is the small additional weight at both ends, which is almost zero, as shown in Fig 5 and Fig 6.

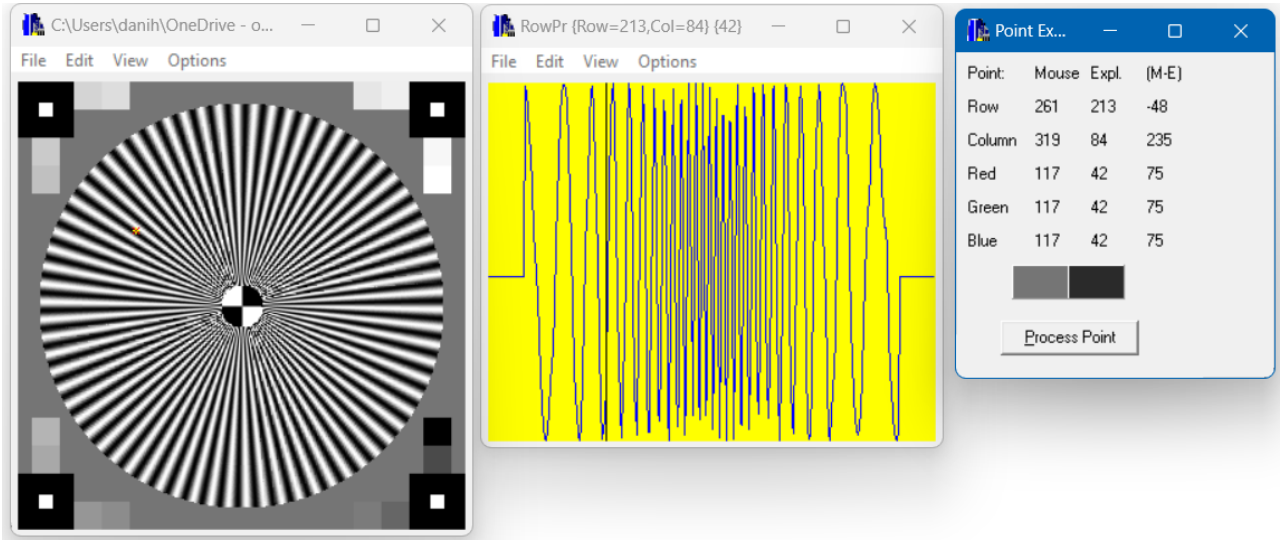


Fig 58: Profilers of Tim2.bmp

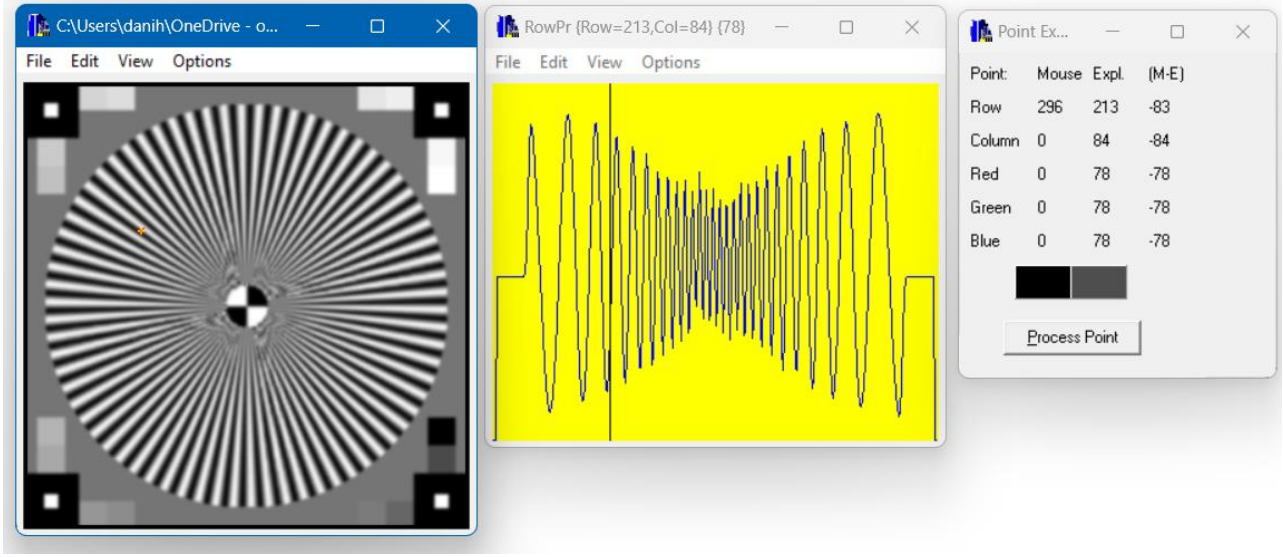


Fig 59: Profilers of Tim2LPF7c.bmp

MP2.23. Blur Image By Using FD Gaussian Filter and FFT

- We designed a Gaussian filter to blur the image Tim2.bmp, as we did during the initial stage of the project , but with a different sigma: sigmaRadius=40.
- The Gaussian filter in Frequency Domain (Fig 60): The Gaussian filter is a low-pass filter that attenuates high frequencies while preserving low frequencies.
- In the frequency domain, it's represented as a Gaussian-shaped function (Fig 61).

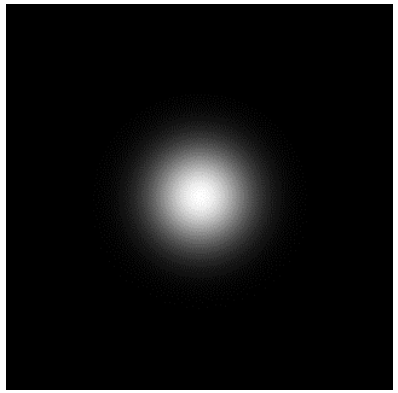


Fig 60: Gaussian Filter in FD

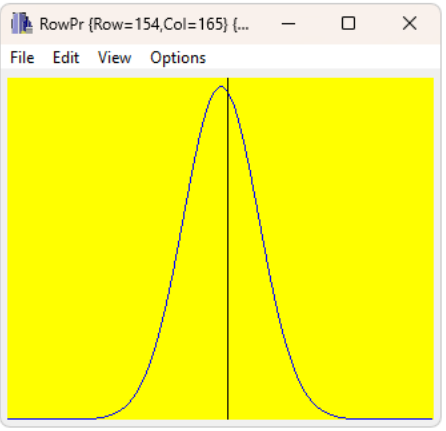


Fig 61: Profilers of Gaussian Filter in FD

MP2.23. Blur Image By Using FD Gaussian Filter and FFT

- We applied FD Gaussian filter over the original Tim1.bmp image (Fig 61) by using convolution.
- Applying a Gaussian low-pass filter (LPF) in the frequency domain (FD) typically results in attenuating higher frequency components of the signal while preserving the lower frequency components.
- This smoothing effect helps reduce noise and sharp transitions in the signal, resulting in a smoother and more regular frequency spectrum (Fig 63).
- Comparing this FFT blurred image (Fig 63) with the previous LPF blurred images (Fig 57, 59), it is easy to see a significant difference. This image is more blurred, and the row profiler shows much more smooth transition in pixel values and dumped amplitudes.

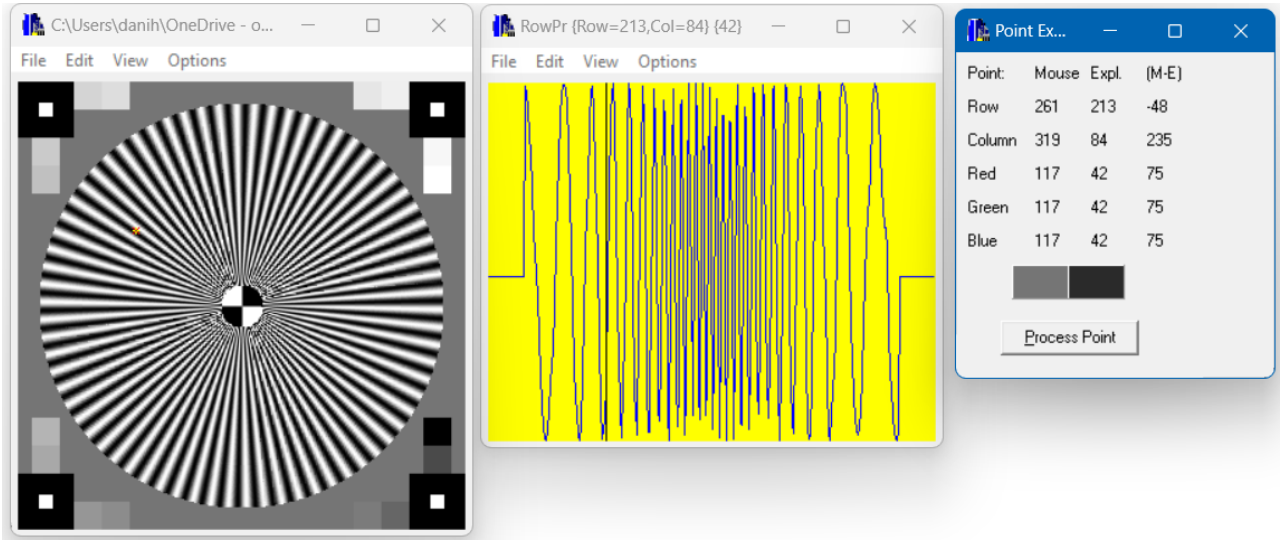


Fig 62: Profilers of Tim2.bmp

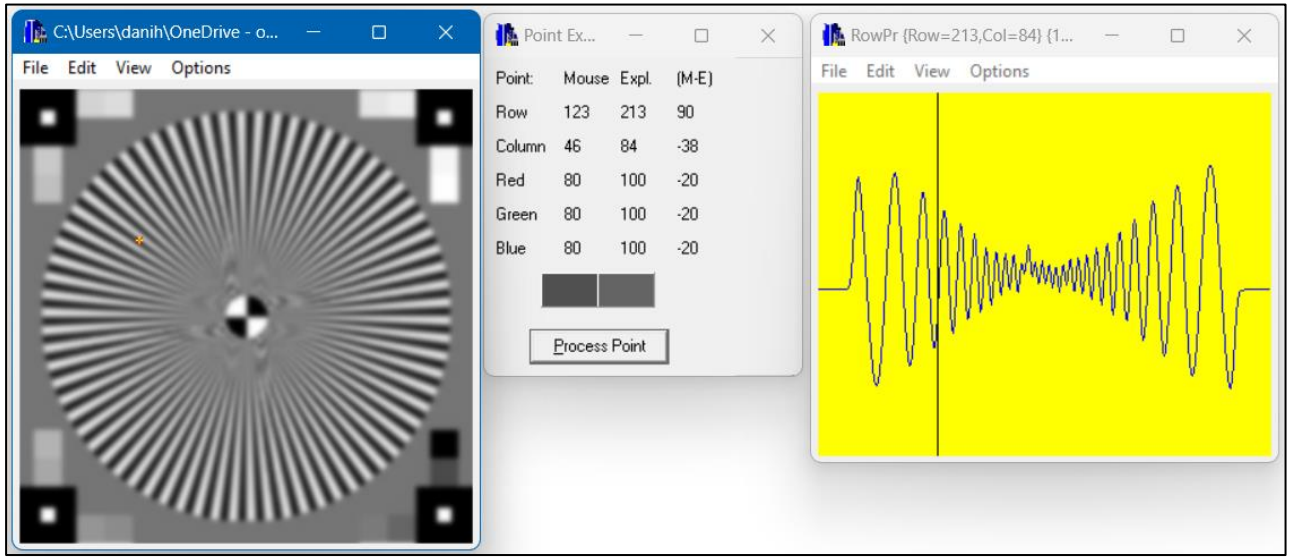


Fig 63: Profilers of Tim2LPFf.bmp

MP2.24. Restore Blurred Images – Convolution – HPF

- We applied the same high-pass convolution filters that we had designed before.
- Both filters are sharpening filters.
- Sharpening enhances image details and edges, making them more prominent.
- It involves convolving the image with a sharpening kernel, which emphasizes high-contrast areas

HPF1c Design: $\begin{bmatrix} -2 & -2 & -2 \\ -2 & 17 & -2 \\ -2 & -2 & -2 \end{bmatrix}$

HPF2c Design: $\frac{1}{16} * \begin{bmatrix} -8 & -16 & -8 \\ -16 & 112 & -16 \\ -8 & -16 & -8 \end{bmatrix}$

MP2.25. Restore Blurred Images – FFT

- We created a high-pass filter Inversed Gaussian filter in the frequency domain (FD- Fig 64) with sigmaRadius = 40.
- This filter acts as the inverse of the original Gaussian LPF in the frequency domain as described in slide MP2.24 (Fig 65).
- Our aim with this filter was to effectively restore the Gaussian filter in the frequency domain (FD). Multiplying the filter and its inverse in FD results in the identity filter, achieving nearly perfect restoration.

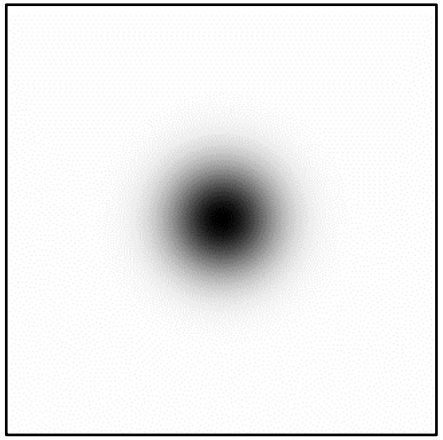


Fig 64: Gaussian Filter in FD

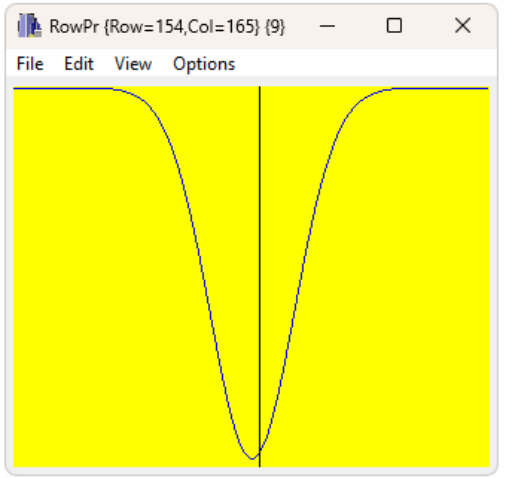


Fig 65: Profilers of Gaussian Filter in FD

MP2.26. Restore Blurred Images – Results

- As done before, we utilized the set of three High Pass Filters (HPFs) that we designed specifically for this task to enhance the clarity and detail of the three blurred images.
- Taking a systematic approach, we systematically applied each HPF to every individual blurred image in the set.
- Afterward, we carefully analyzed the effects and improvements brought about by each HPF on the respective blurred image it was applied to.
- This comprehensive evaluation allowed us to gain insights into the efficacy and performance of each HPF in terms of enhancing image sharpness and detail.

MP2.26. Restore Blurred Images – Results – HPF1c

- We utilized HPF1c on the blurred image Tim2LPF5c.bmp (Fig 66).
- In Fig 67 we can see the restored image using the first HPF. The image appears to have high-frequency noise, likely due to the application of the HPF. The radial lines seem to be more pronounced compared to the first blurred image, suggesting some level of restoration. However, the image still lacks the clarity of the original image. It also appears slightly more saturated.
- The restored image shows some improvement in detail compared to the first blurred image, but it does not fully match the clarity of the original image. This is expected as once an image is blurred (high-frequency details are lost), it's challenging to fully restore it to its original state using a HPF.
- The row profiler matches the above. It changes more drastically compared to the original image, as can be seen by the capping at the top and bottom.

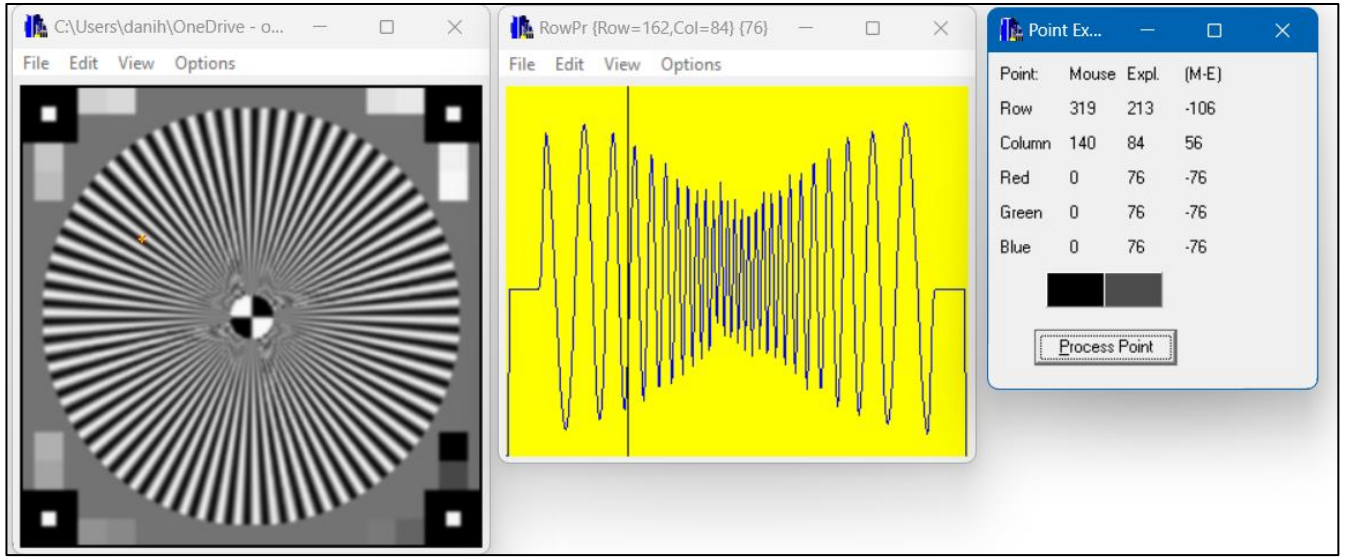


Fig 66: Profilers for Tim2LPF5c.bmp

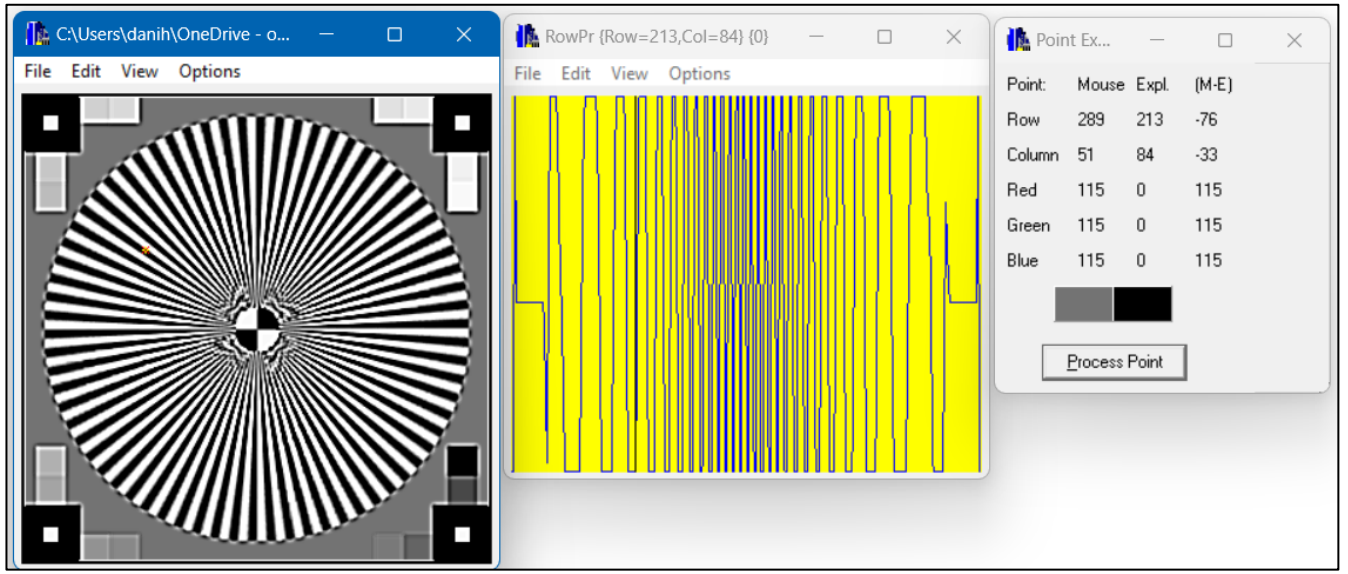


Fig 67: Profilers for HPF1cTim2LPF5c.bmp

MP2.26. Restore Blurred Images – Results – HPF1c

- We utilized HPF1c on the blurred image Tim2LPF7c.bmp (Fig 68).
- In Fig 69 we can see the restored image using the first HPF. Again, The image appears to have high-frequency noise, likely due to the application of the HPF.
- The restored image shows some improvement in detail compared to the first blurred image, but it does not fully match the clarity of the original image in this case either.

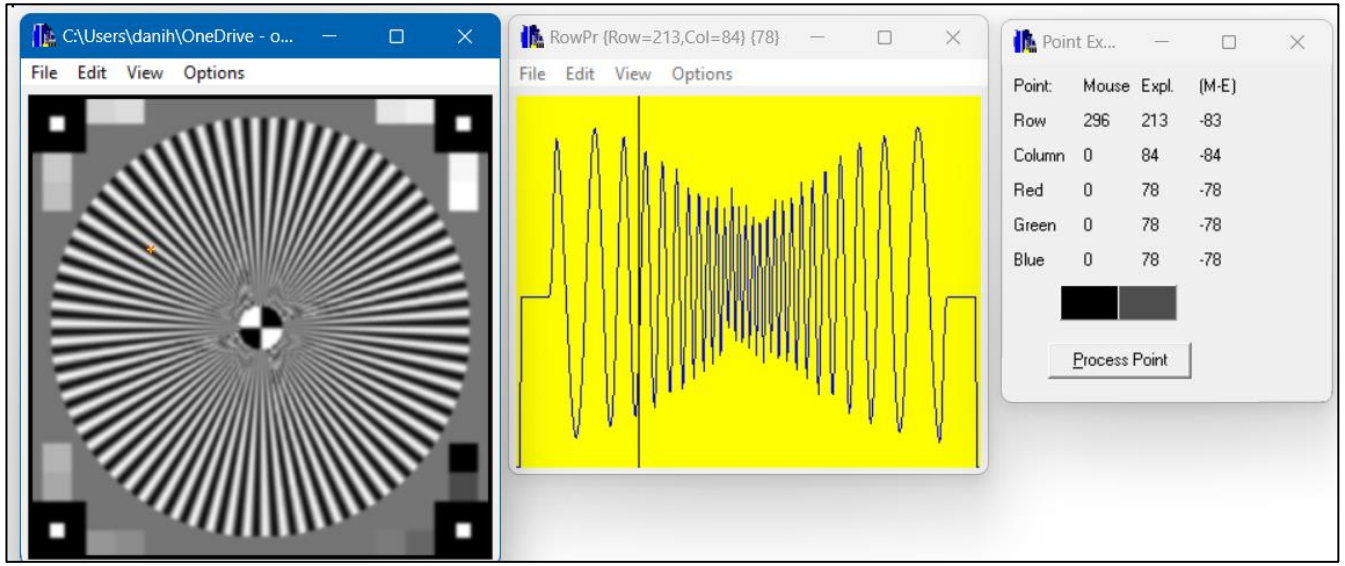


Fig 68: Profilers for Tim2LPF7c.bmp

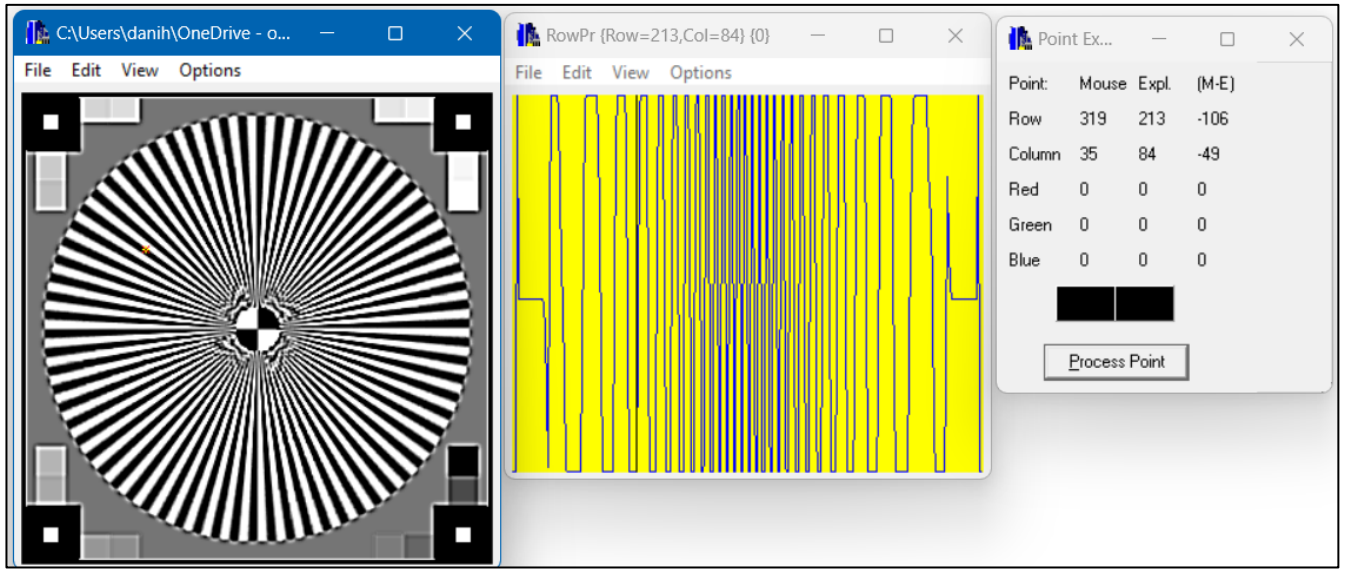


Fig 69: Profilers for HPF1cTim2LPF7c.bmp

MP2.26. Restore Blurred Images – Results – HPF1c

- We utilized HPF1c on the blurred image Tim2LPFf.bmp (Fig 70).
- In Fig 71 we can see the restored image using the first HPF. The HPF is effective in restoring the original details of the image. Although there is some high-frequency noise, the image appears sharp and clear.
- The row profiler better resembles the original image's row profile, albeit the amplitude is dumped in the middle. The is because losing the high frequency components during the blur process.

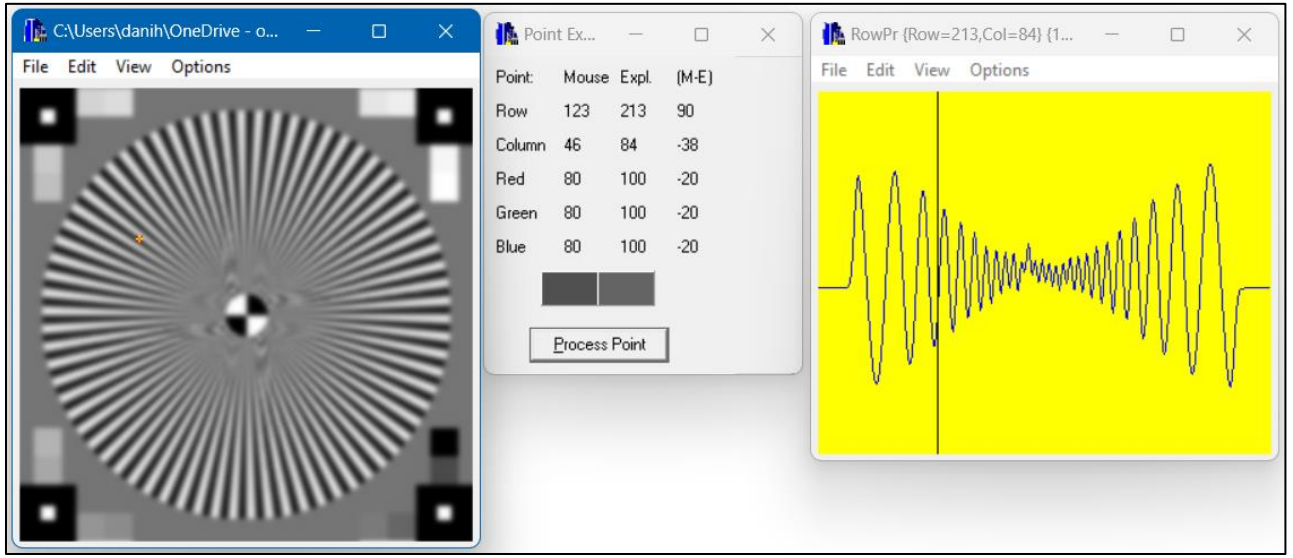


Fig 70: Profilers for Tim2LPFf.bmp

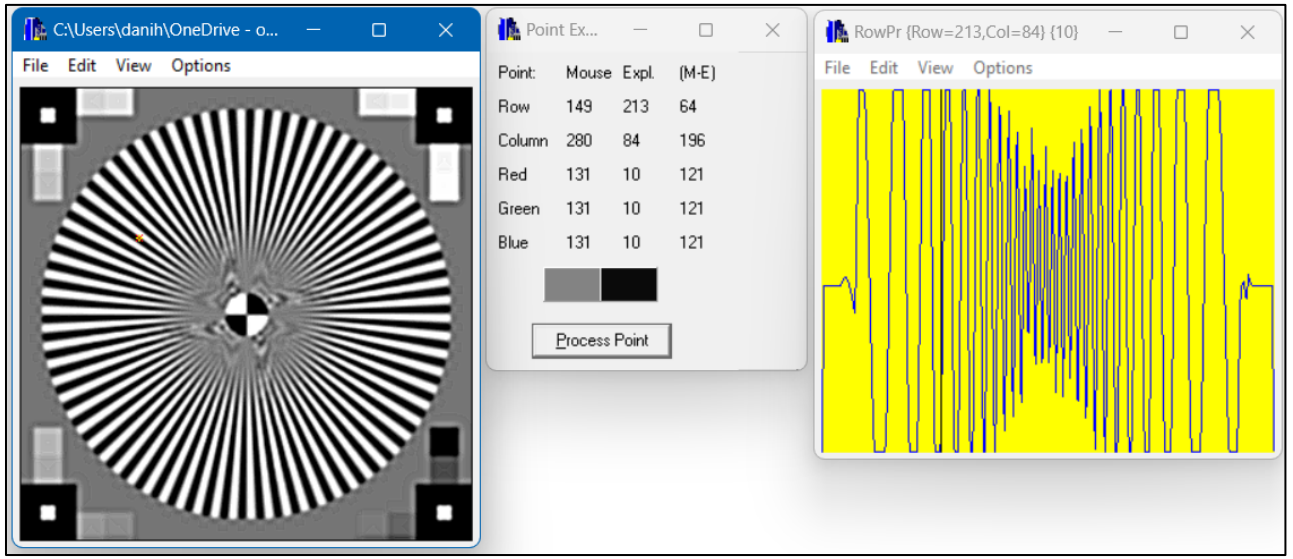


Fig 71: Profilers for HPF1cTim2LPFf.bmp

MP2.26. Restore Blurred Images – Results – HPF2c

- We utilized HPF2c on the blurred image Tim2LPF5c.bmp (Fig 72).
- In Fig 73 we can see the restored image using the second HPF. The HPF used here was slightly more effective in restoring the original details of the image. There is less high-frequency noise, and the image better maintains the sharpness and clarity of the original image.
- Also, the row profilers seems to change less rapidly.

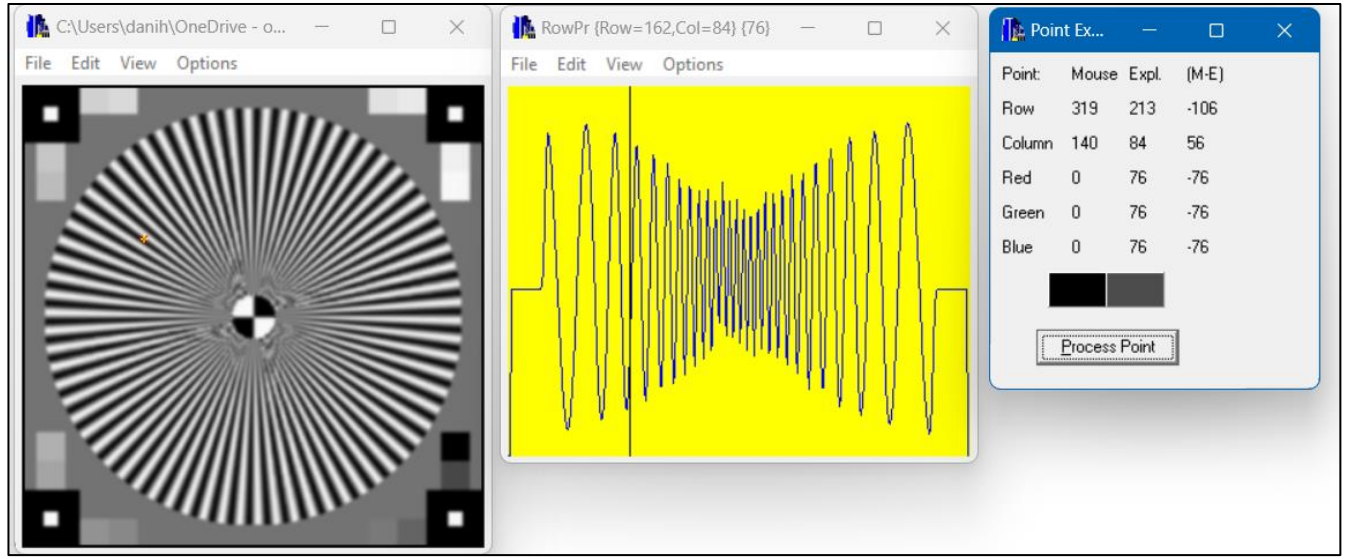


Fig 72: Profilers for Tim2LPF5c.bmp

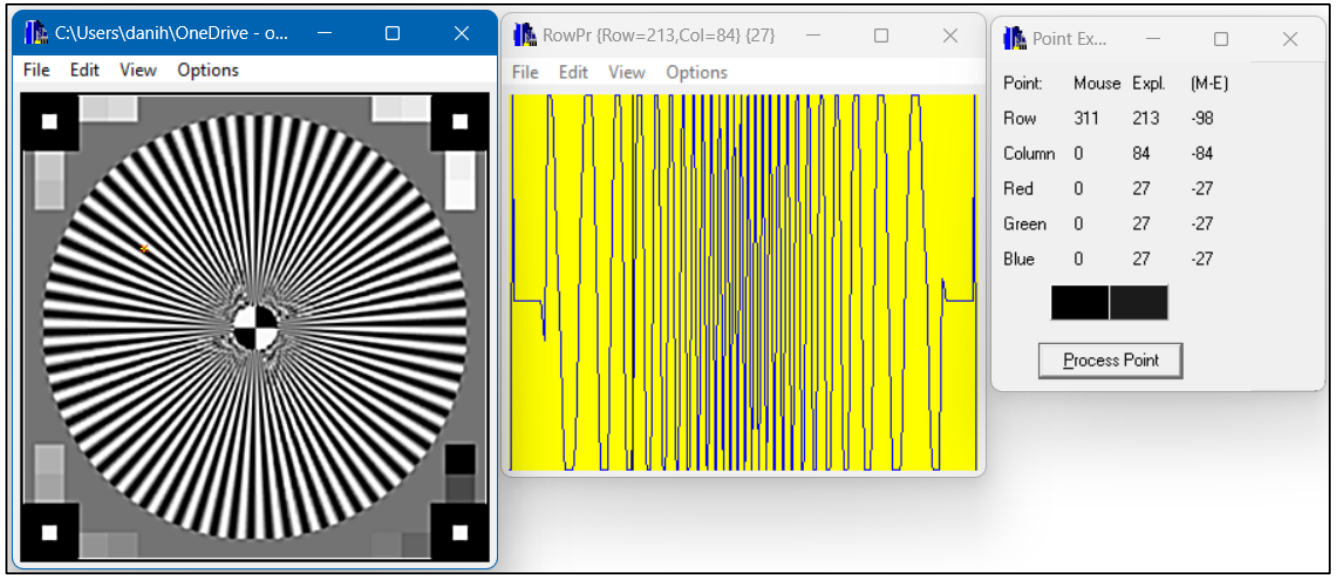


Fig 73: Profilers for HPF2cTim2LPF5c.bmp

MP2.26. Restore Blurred Images – Results – HPF2c

- We utilized HPF2c on the blurred image Tim2LPF7c.bmp (Fig 74).
- In Fig 75 we can see the restored image using the second HPF. This HPF was again more effective in restoring the original details of the image. There is less high-frequency noise, and the image better maintains the sharpness and clarity of the original image.
- Again, the row profilers seems to change less rapidly.

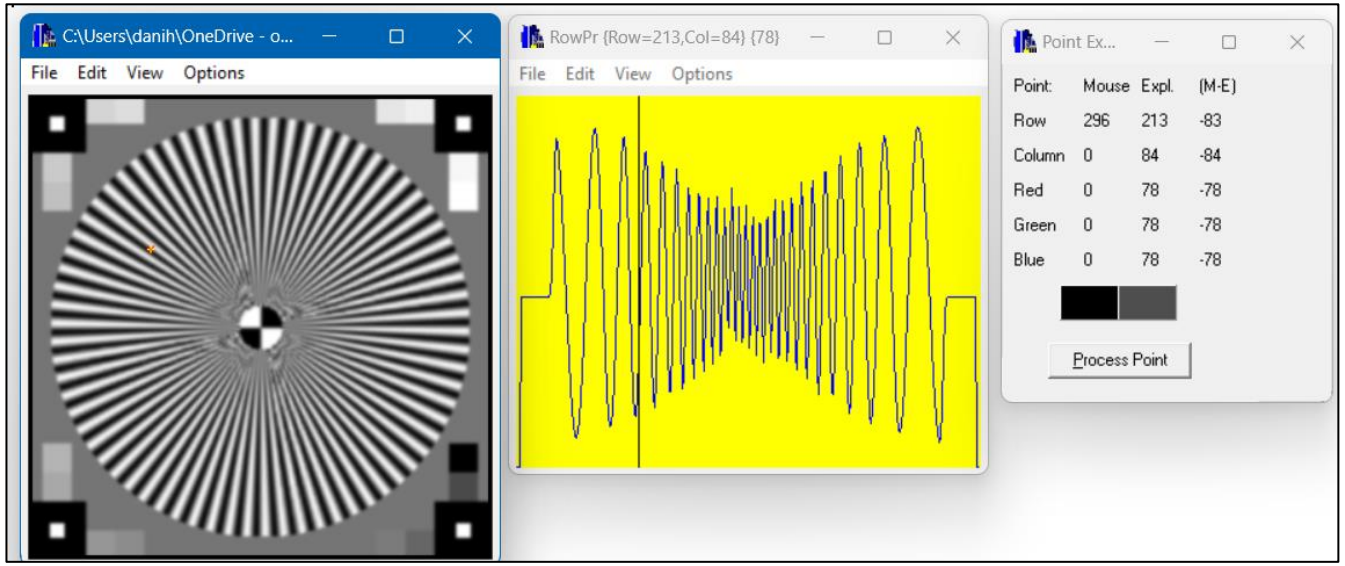


Fig 74: Profilers for Tim2LPF7c.bmp

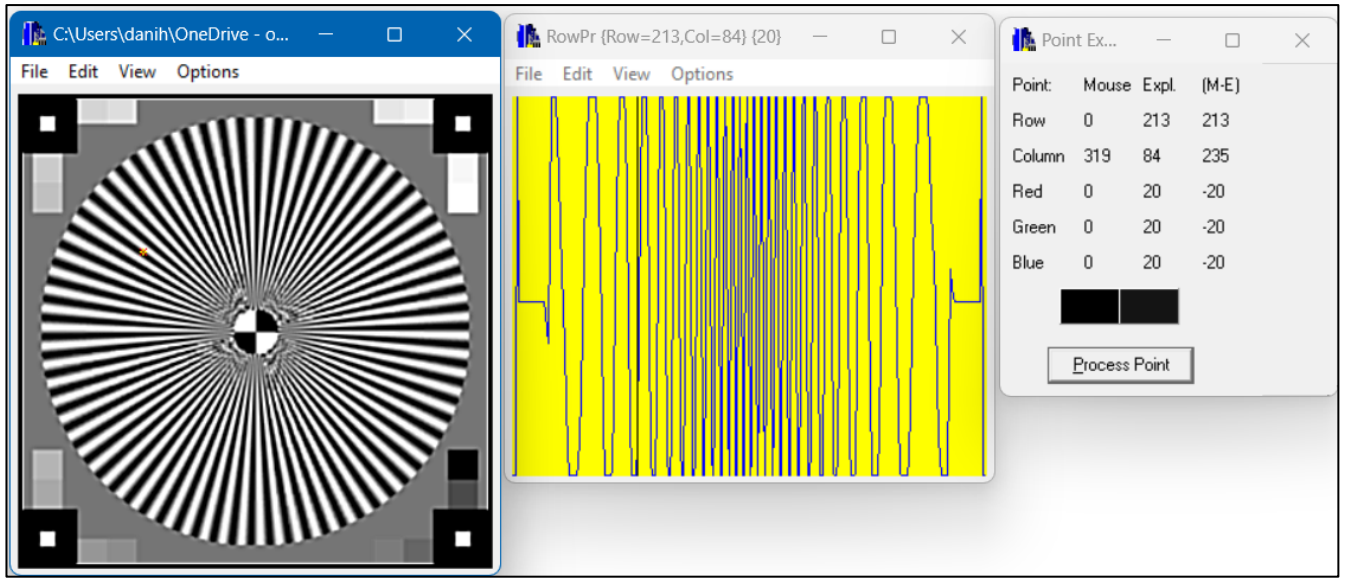


Fig 75: Profilers for HPF2cTim2LPF7c.bmp

MP2.26. Restore Blurred Images – Results – HPF2c

- We utilized HPF2c on the blurred image Tim2LPFf.bmp (Fig 76).
- In Fig 77 we can see the restored image using the second HPF. This HPF is **less** effective in restoring the original details of the image. The image is still blurry. Albeit less blurry, but it doesn't match the original image in its sharpness and clarity.
- The row profiler of the restored image better resembles the original row profile compared to the blurred one but is still dumped in the middle.

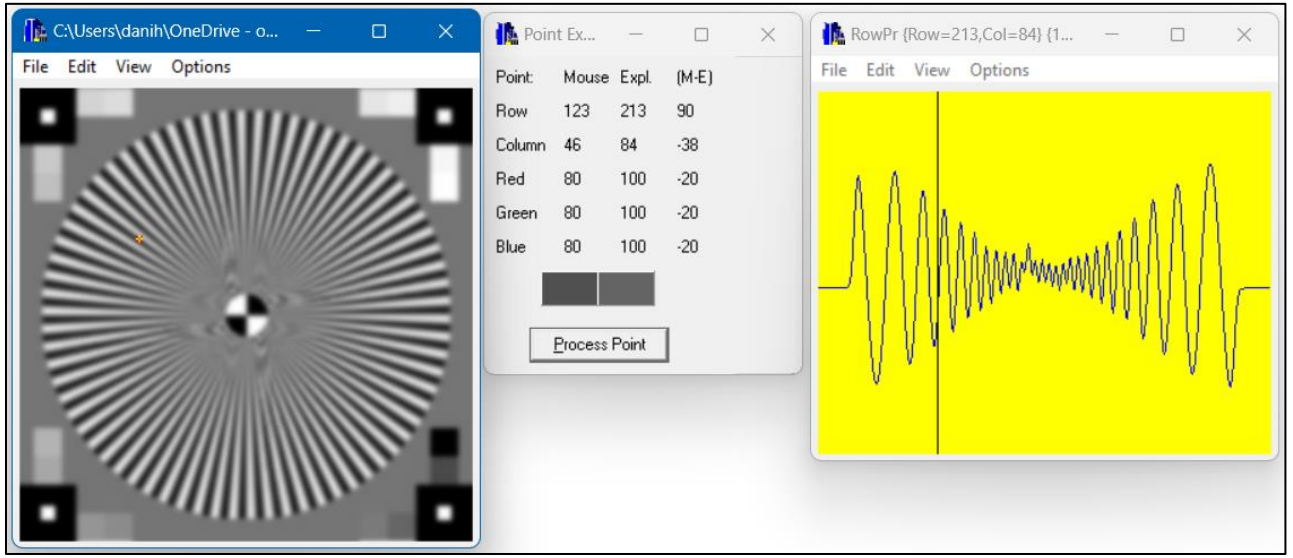


Fig 76: Profilers for Tim2LPFf.bmp

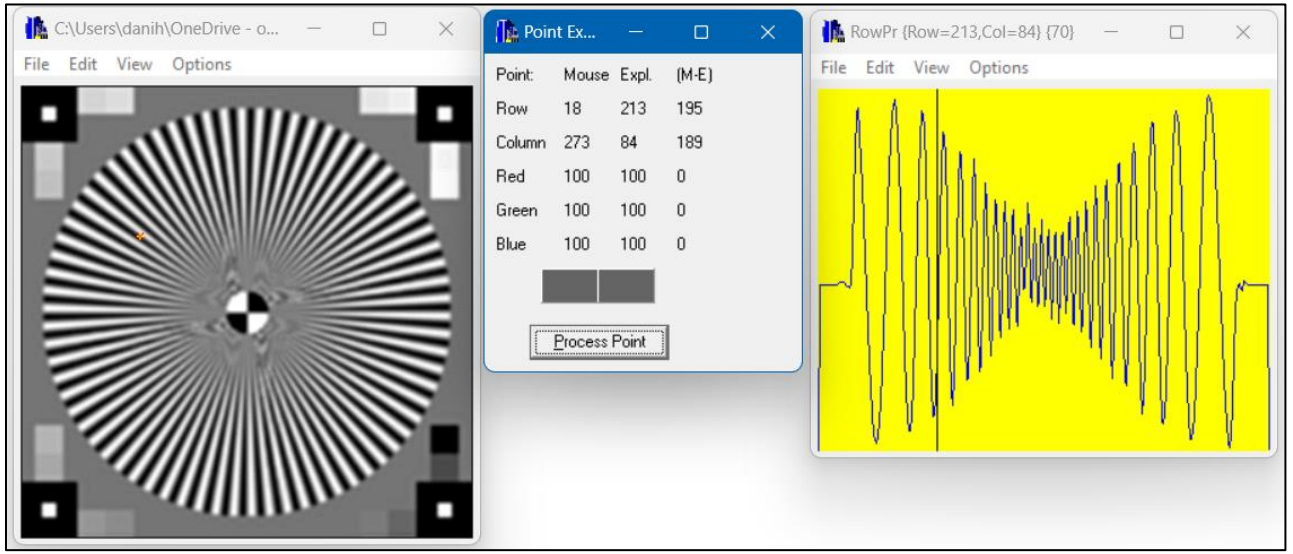


Fig 77: Profilers for HPF2cTim2LPFf.bmp

MP2.26. Restore Blurred Images – Results – HPFf

- We utilized HPFf on the blurred image Tim2LPF5c.bmp (Fig 78).
- In Fig 79 we can see an attempt at restoring the blurred image using FFT. It doesn't seem sharper, and the image is more grayscale and less vibrant.
- It's important to note that while FFT can help recover some of the lost details from the blurring process, it may not perfectly restore the image to its original state. This is because some information might have been lost during the blurring process that cannot be recovered.
- The row profiler of the restored image doesn't quite resemble the original row profile either. It has the same pattern (increasing oscillation towards the middle), but the amplitude is smaller.

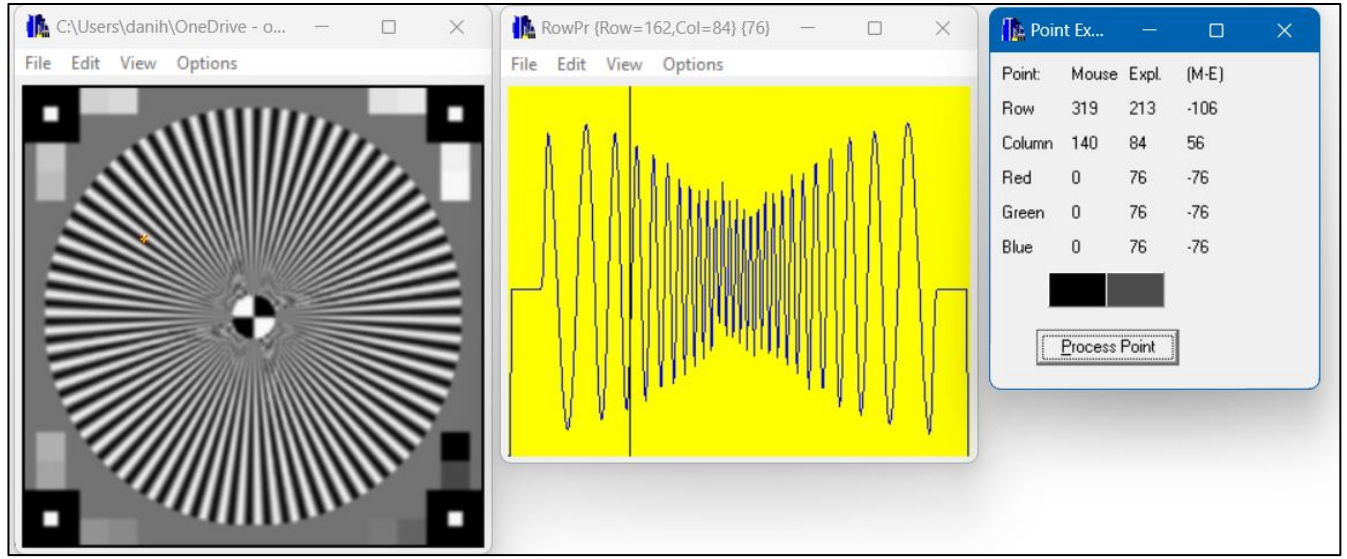


Fig 78: Profilers for Tim2LPF5c.bmp

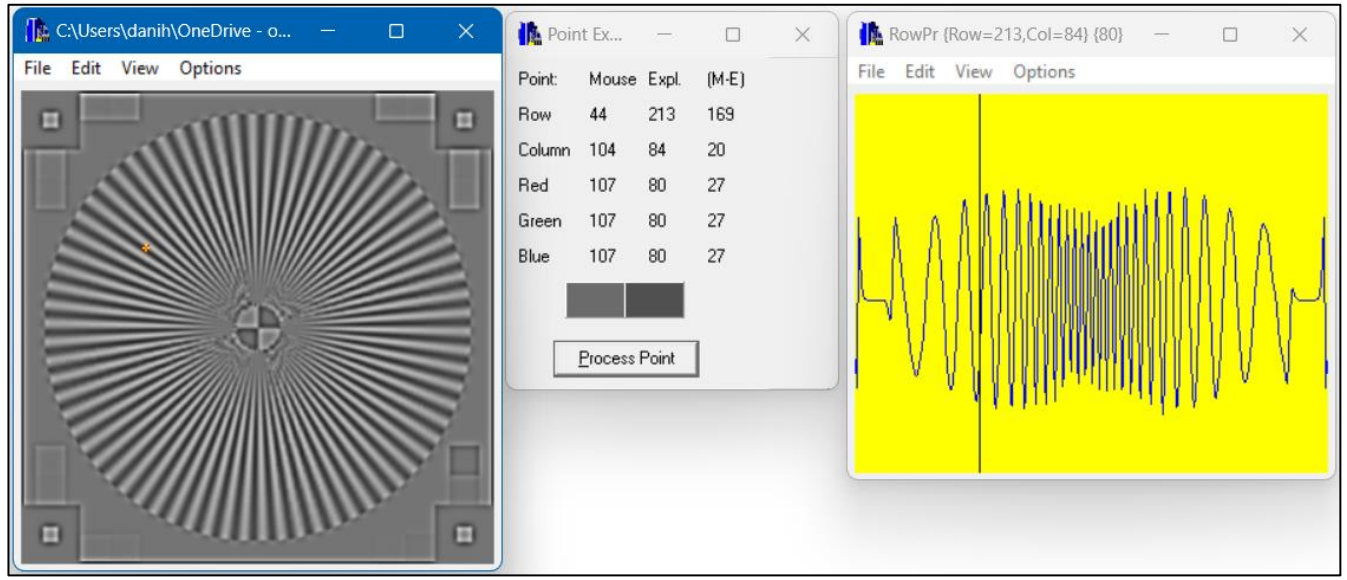


Fig 79: Profilers for HPFfTim2LPF5c.bmp

MP2.26. Restore Blurred Images – Results – HPFf

- We utilized HPFf on the blurred image Tim2LPF7c.bmp (Fig 80).
- In Fig 81 we can see an attempt at restoring the blurred image using FFT. As expected, because the 2 blurred images look the same, also the restored image doesn't look too different from the previous one.
- As previously the restored image doesn't seem sharper, and the image is more grayscale and less vibrant.
- The row profiler of the restored image doesn't quite resemble the original row profile. It has the same pattern (increasing oscillation towards the middle), but the amplitude is smaller.

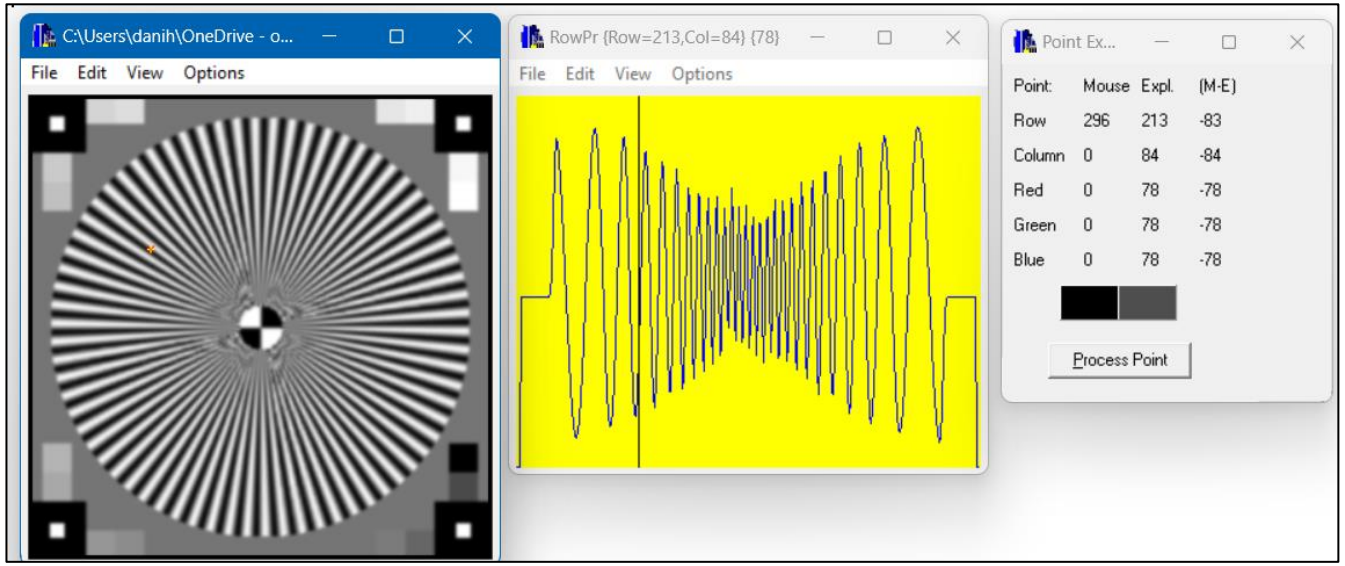


Fig 80: Profilers for Tim2LPF7c.bmp

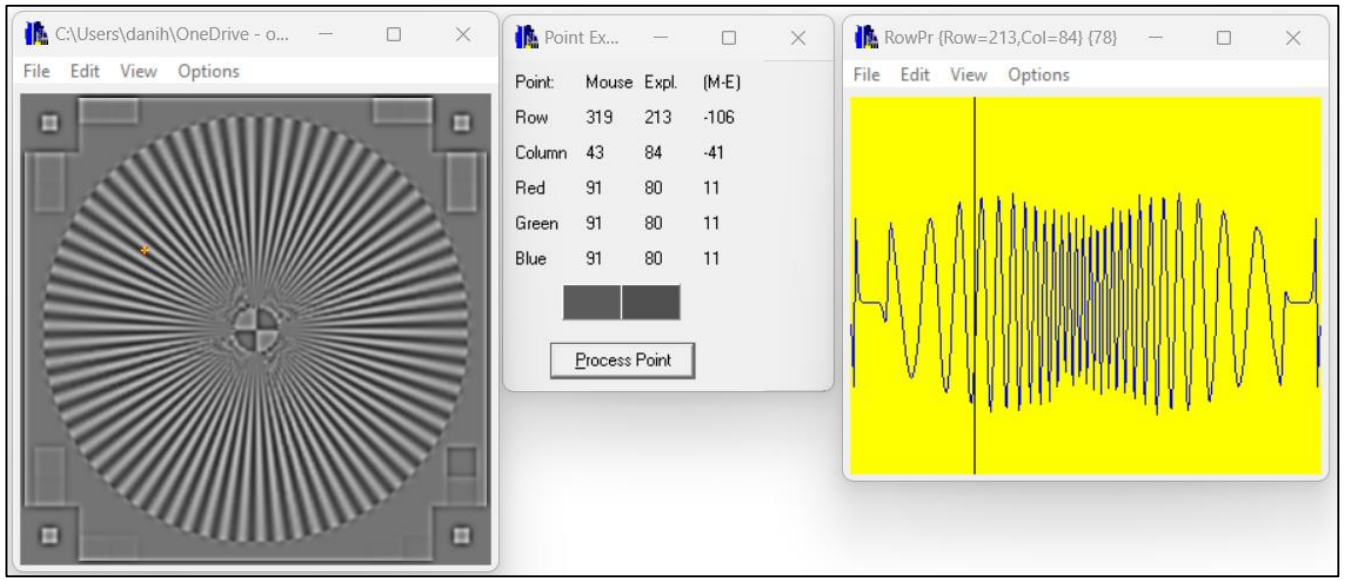


Fig 81: Profilers for HPFfTim2LPF7c.bmp

MP2.26. Restore Blurred Images – Results – HPFf

- We utilized HPFf on the blurred image Tim2LPFf.bmp (Fig 82).
- In Fig 83 we can see an attempt at restoring the blurred image using FFT. It seems sharper than the blurred image but not as sharp as the original. Also, it appears more gray and less vibrant. It did a decent job at restoring the image, but it's far from perfect.
- The row profiler of the restored image doesn't quite resemble the original row profile either. It oscillates but the amplitude doesn't match the original image's amplitudes.

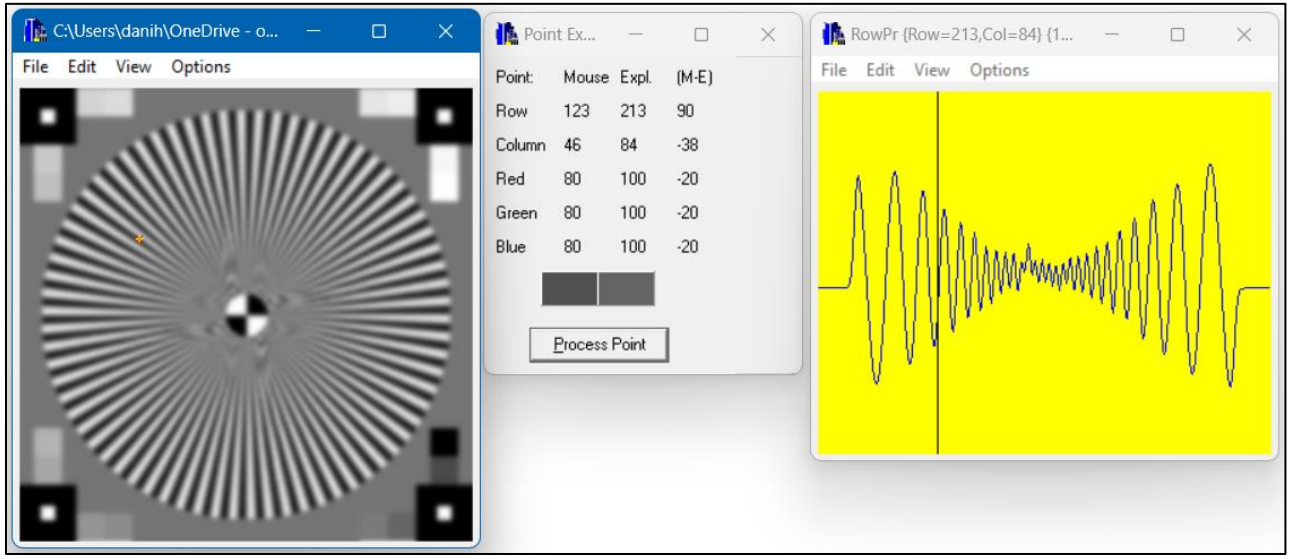


Fig 82: Profilers for Tim2LPFf.bmp

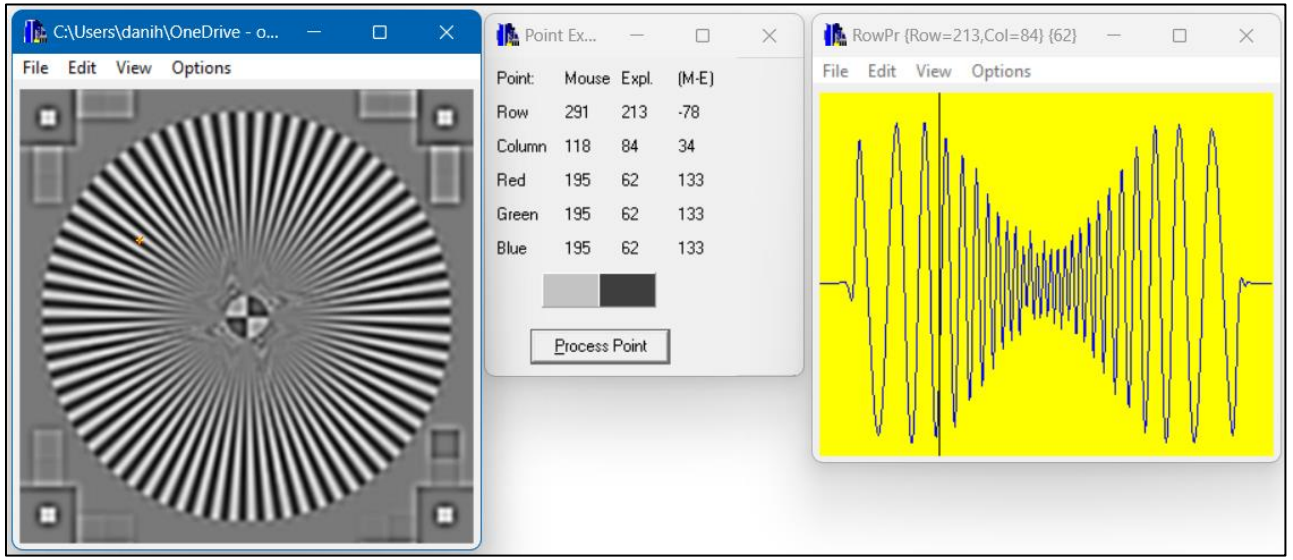
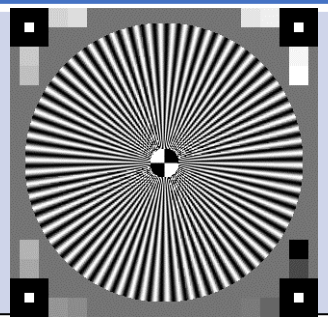
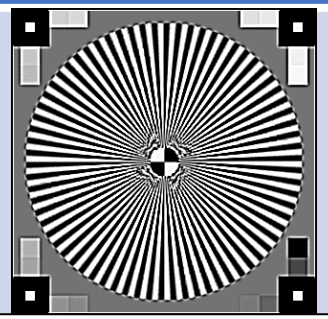
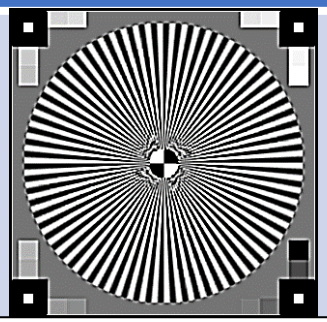
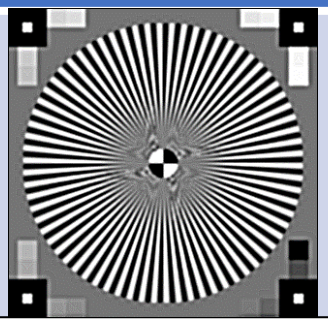
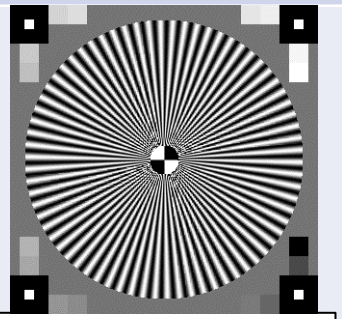
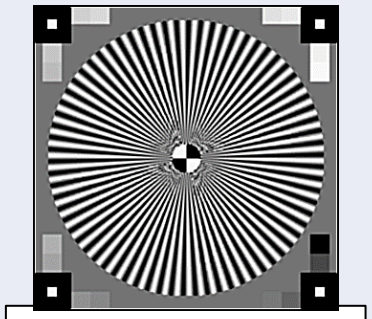
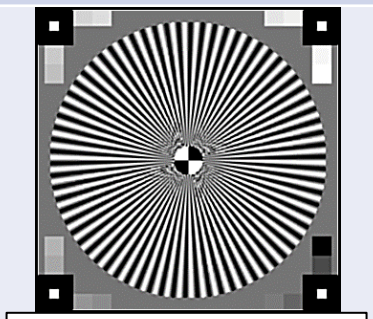
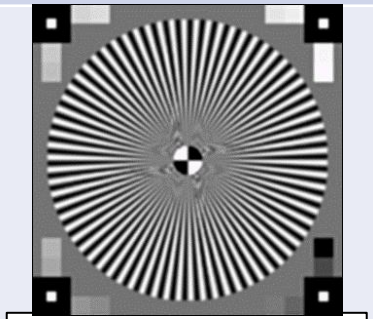
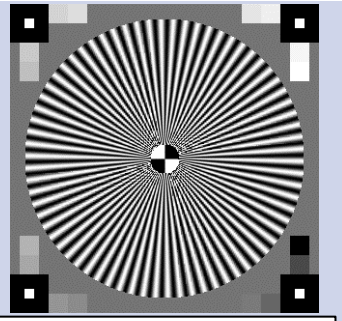
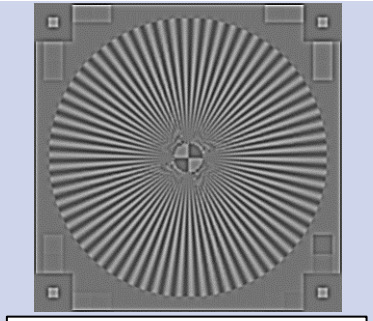
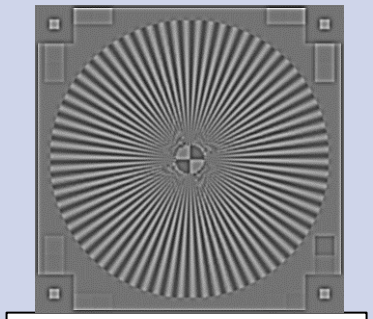
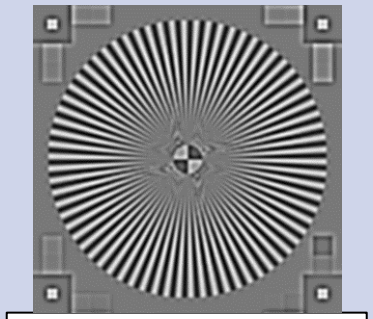


Fig 83: Profilers for HPFfTim2LPFf.bmp

MP2.26. Restore Blurred Images – Discussion

HPF Used	Original Image	Restored Blurred Image (After LPF5c)	Restored Blurred Image (After LPF7c)	Restored Blurred Image (After LPFf)
HPF1c	 Fig 84: Tim2.bmp	 Fig 85: HPF1cTim2LPF5c.bmp	 Fig 86: HPF1cTim2LPF7c.bmp	 Fig 87: HPF1cTim2LPFf.bmp
HPF2c	 Fig 88: Tim2.bmp	 Fig 89: HPF2cTim2LPF5c.bmp	 Fig 90: HPF2cTim2LPF7c.bmp	 Fig 91: HPF2cTim2LPFf.bmp
HPFf	 Fig 92: Tim2.bmp	 Fig 93: HPFfTim2LPF5c.bmp	 Fig 94: HPFfTim2LPF7c.bmp	 Fig 95: HPFfTim2LPFf.bmp

- We can see that the attempts at restoring the blurred image using FFT are far from perfect.
- The darker appearance of the background in the resulting image is due to the blurring effect, which reduces contrast and lead to the loss of high-frequency information contributing to the background's brightness.
- The results of restored blurred images after LPF5c and LPF7c (Figs 85-86 for example) are very similar because the only distinction is the size of the Low Pass Filters (LPFs), which are essentially the same except for their sizes.

MP2.3. Main Code

```
// Applies Fast Fourier Transform (FFT) to the blurred images specified
// by the input filenames and saves the filtered images.
void ApplyFFTToBlurredImages(vector<string> inputFileNames, int radius)
{
    int fileNamesCount = inputFileNames.size();

    for (int i = 0; i < fileNamesCount; i++)
    {
        c2dFloatGrayImage imageRe = ApplyFFT(inputFileNames[i], false, true, radius);
        string outputFileName = AddBMPExtention("HPFf" + inputFileNames[i]);
        imageRe.SaveAsOptimalGrayBmpFile(outputFileName.c_str());
    }
}
```

Fig 96: Apply FFT to all 3 blurred images

```
//Applies High Pass Filters (HPF) to the blurred images specified by the input filename
void ApplyHPFToBlurredImages(vector<string> inputFileNames)
{
    // Define High Pass Filter kernels
    double HPF1[3][3] = { {-2, -2, -2}, {-2, 17, -2}, {-2, -2, -2} };
    double HPF2[3][3] = { {-8, -16 , -8}, {-16, 112 , -16}, {-8, -16 , -8} };
    // Define normalization factor for HPF2
    double HTP2factor = 1.0 / 16.0;
    int fileNamesCount = inputFileNames.size();
    for (int i = 0; i < fileNamesCount; i++)
    {
        // Load the input image
        c2dByteGrayImage source(AddBMPExtention(inputFileNames[i]).c_str());
        c2dByteGrayImage dest(source.NumberOfRows(), source.NumberOfColumns());
        // Apply HPF1 to the source image
        DoFiltrationByConvolution(source, dest, HPF1, 1);
        dest.SaveToGrayBmpFile(AddBMPExtention("HPF1c" + inputFileNames[i]).c_str());
        // Apply HPF2 to the source image
        DoFiltrationByConvolution(source, dest, HPF2, HTP2factor);
        dest.SaveToGrayBmpFile(AddBMPExtention("HPF2c" + inputFileNames[i]).c_str());
    }
}
```

Fig 98: Apply Convolution to all 3 blurred images

```
// Applies a Gaussian filter in the frequency domain
//  to the given image and saves the filtered image.
// returns the filtered image name with added "LPFF" suffix
string ApplyFFTAndSave(string inputFileName,
    bool reverseFilter, int radius)
{
    string outputFileName = inputFileName + "LPFF";
    c2dFloatGrayImage imageRe = ApplyFFT(inputFileName, reverseFilter, radius);
    imageRe.SaveAsOptimalGrayBmpFile(AddBMPExtention(outputFileName).c_str());
    return outputFileName;
}
```

Fig 97: Apply Save the processed images after FFT

```
void main()
{
    //Question 2
    cout << "Beginning question 2." << endl;
    string grayImageName = "Tim1";
    vector<string> blurredImagesNames = ApplyFastGaussianFilterConvolution(grayImageName);
    int radiusForTextImage = 80;
    //Question 3
    cout << "Finished question 2. Beginning question 3." << endl;
    blurredImagesNames.push_back(ApplyFFTAndSave(
        grayImageName, false, radiusForTextImage));

    //Question 4
    cout << "Finished question 3. Beginning question 4." << endl;
    ApplyHPFToBlurredImages(blurredImagesNames);

    //Question 5
    cout << "Finished question 4. Beginning question 5." << endl;
    ApplyFFTToBlurredImages(blurredImagesNames, radiusForTextImage);

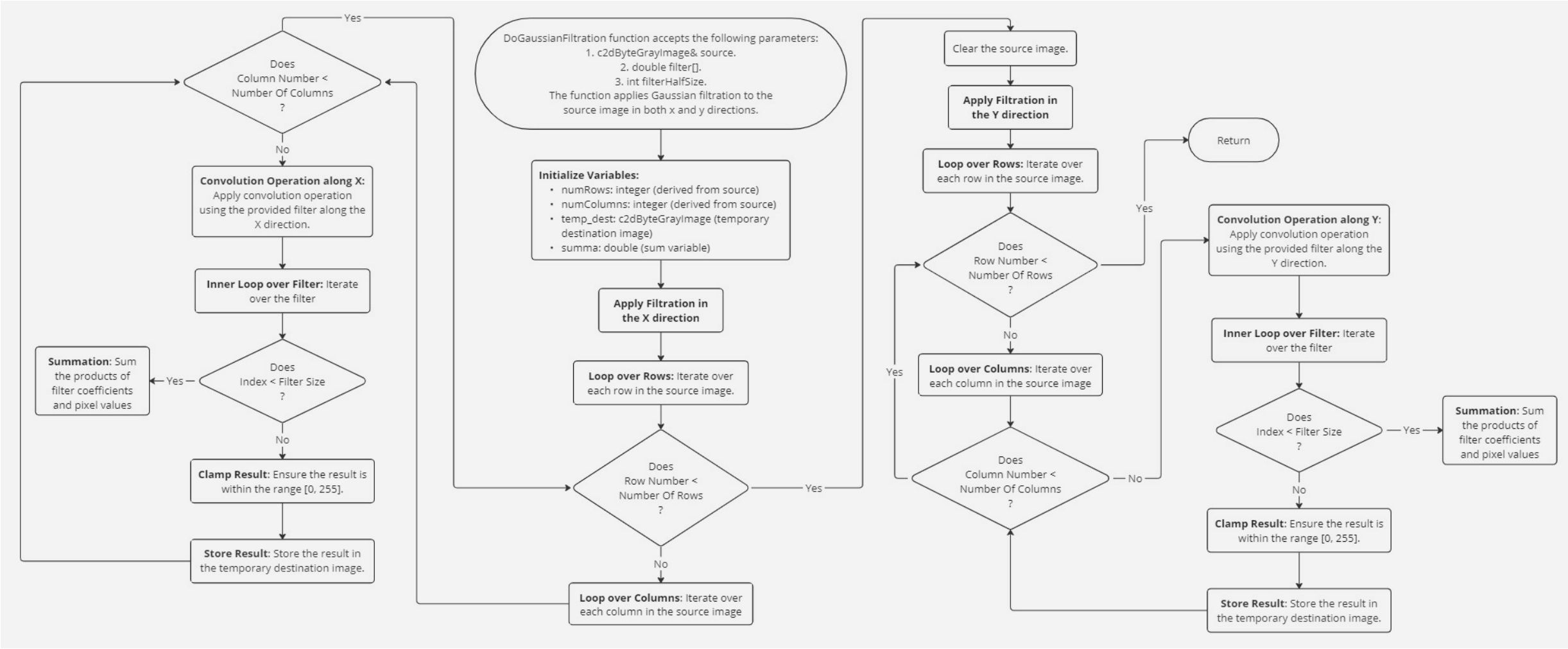
    //Question 7
    cout << "Finished question 5. Beginning question 7." << endl;
    int radiusForTV = 40;
    string TVTestImageName = "Tim2";
    vector<string> blurredTVImagesNames = ApplyFastGaussianFilterConvolution(TVTestImageName);
    blurredTVImagesNames.push_back(ApplyFFTAndSave(TVTestImageName, false, radiusForTV));

    ApplyHPFToBlurredImages(blurredTVImagesNames);
    ApplyFFTToBlurredImages(blurredTVImagesNames, radiusForTV);

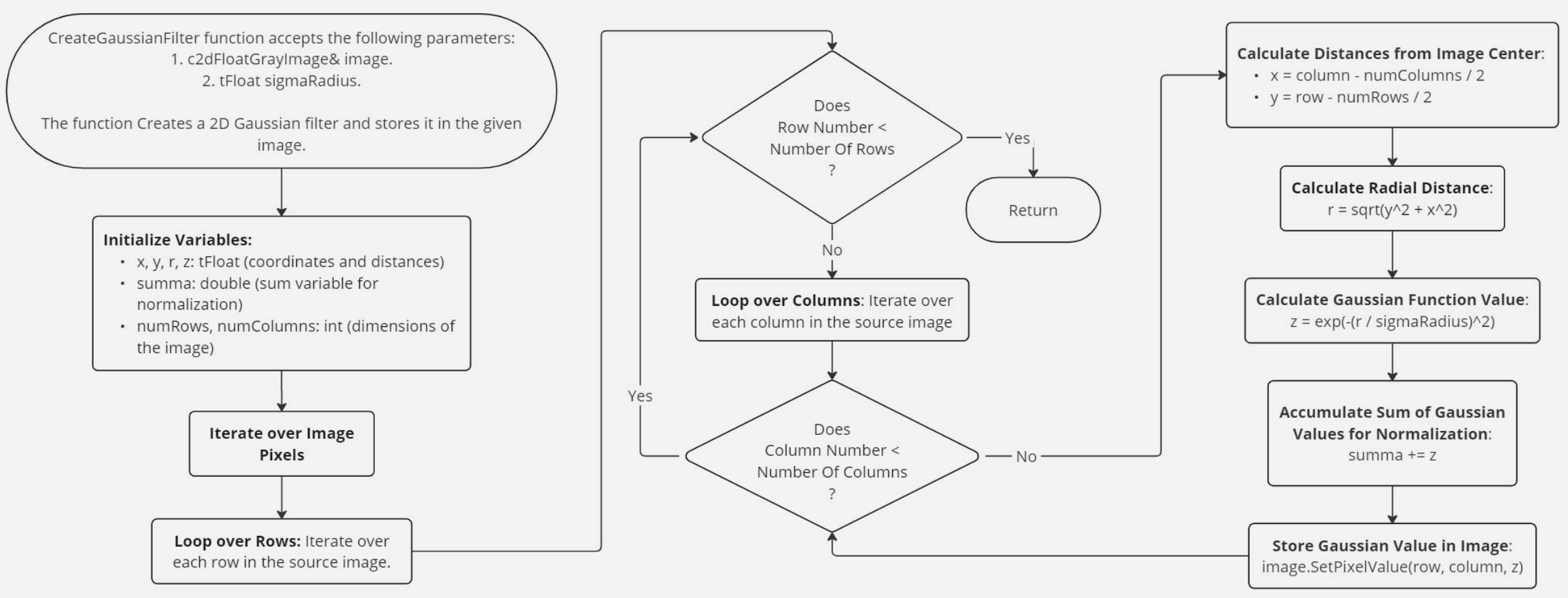
    system("pause");
}
```

Fig 99: Main code

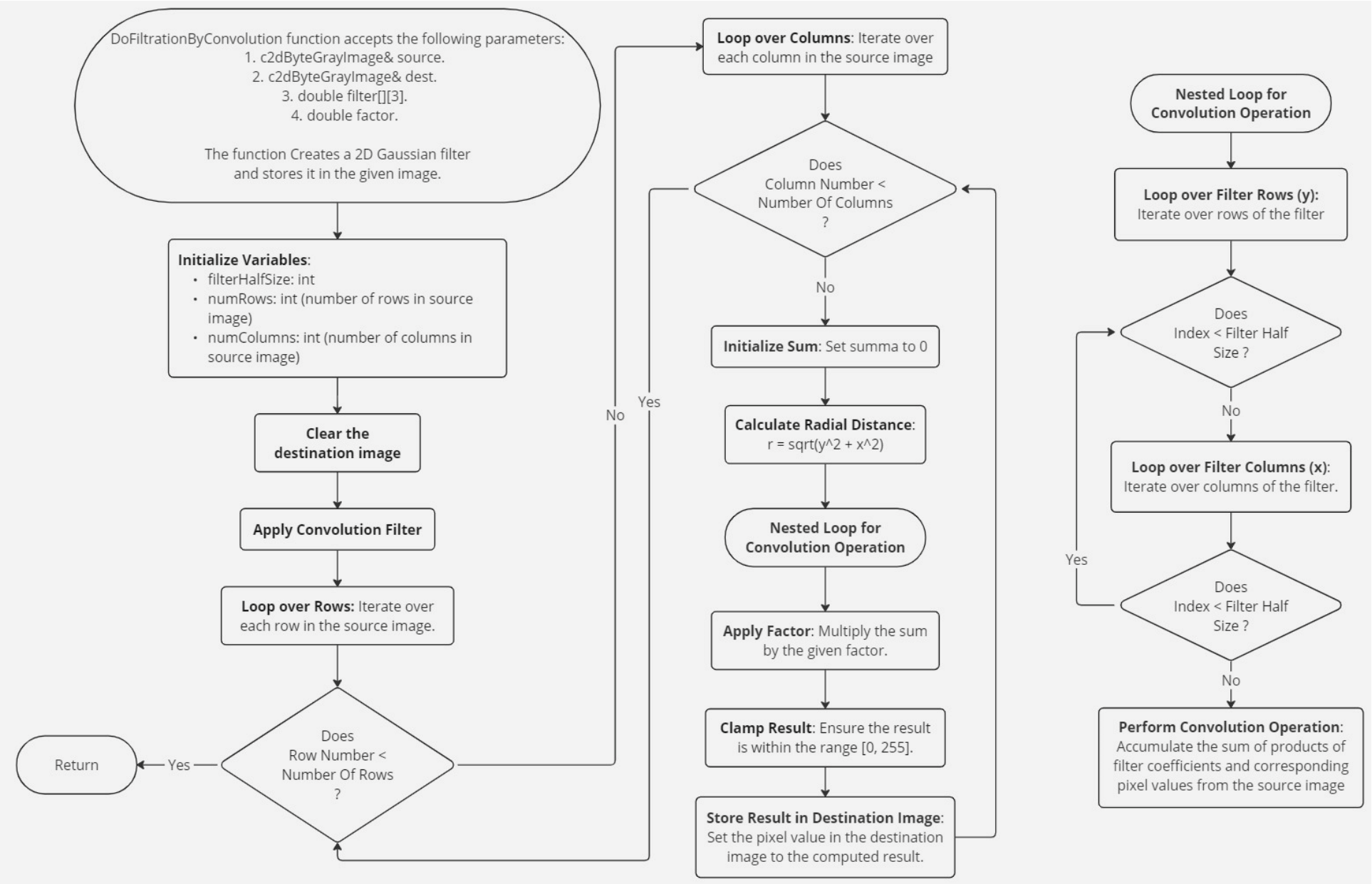
MP2.4. Algorithm's Flow Charts – DoGaussianFiltration



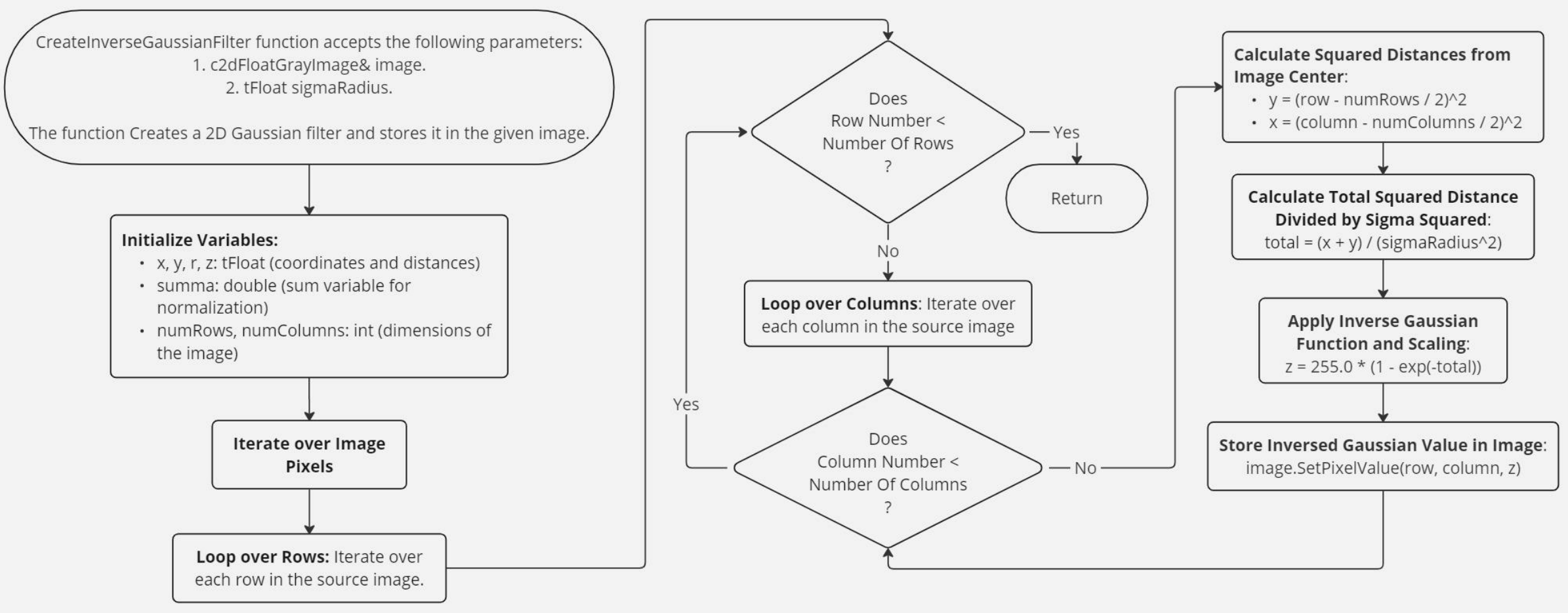
MP2.4. Algorithm's Flow Charts – CreateGaussianFilter



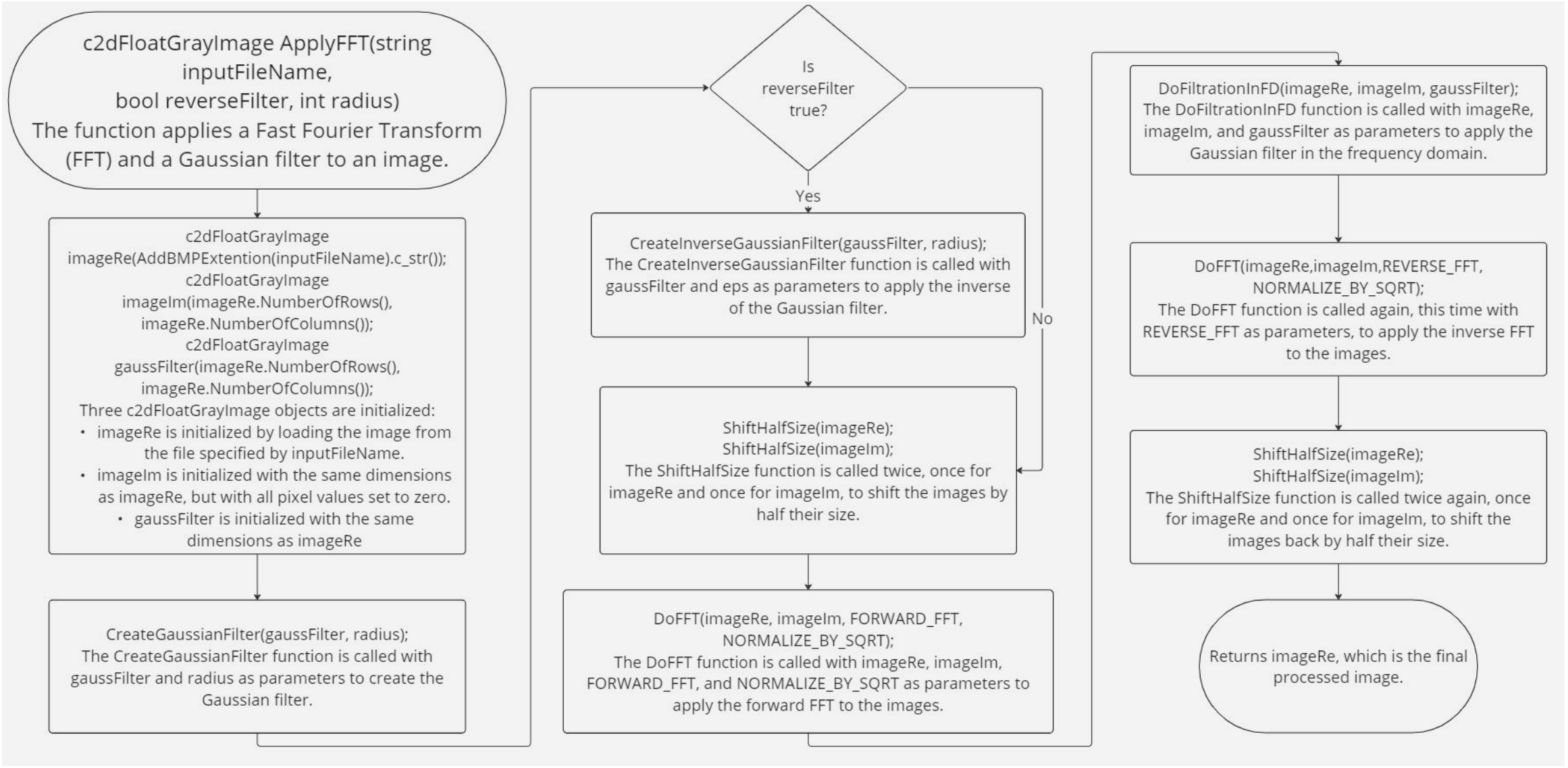
MP2.4. Algorithm's Flow Charts – DoFiltrationByConvolution



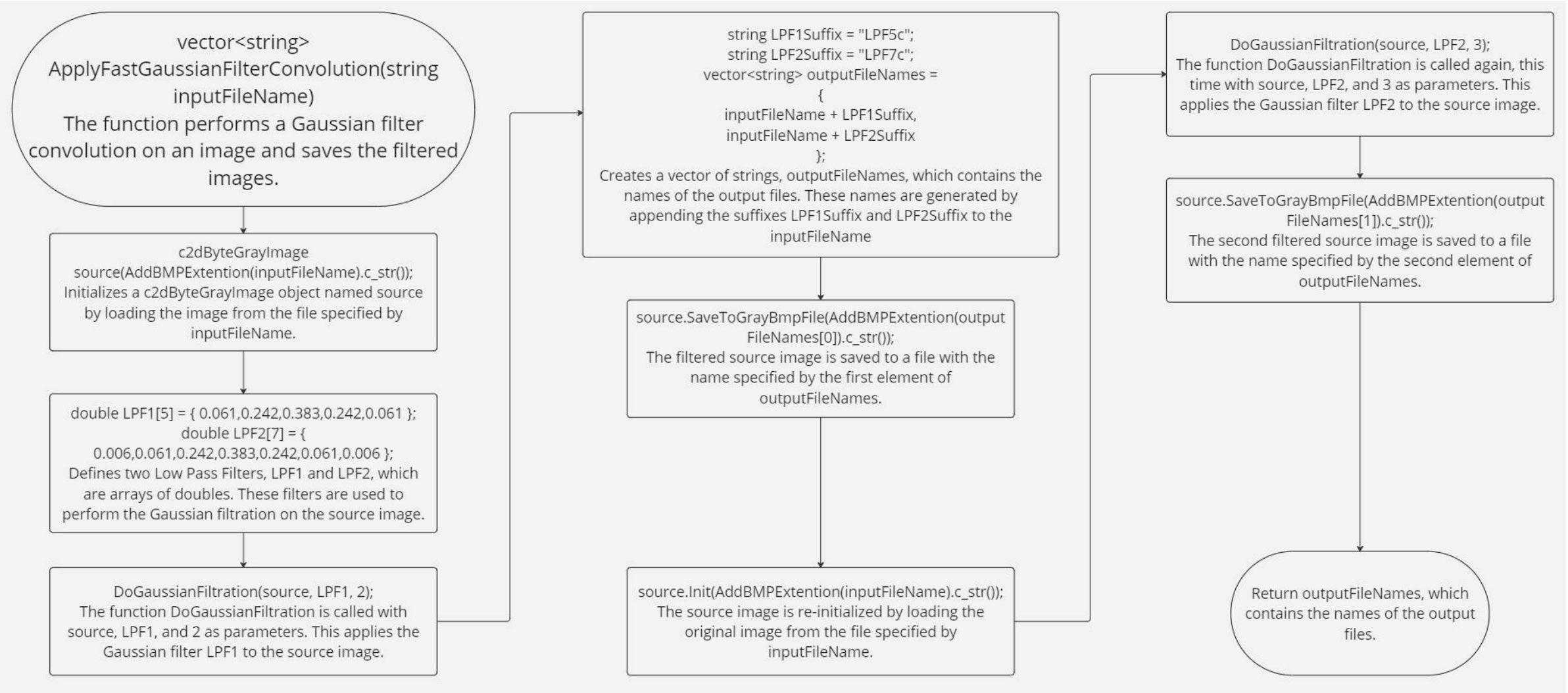
MP2.4. Algorithm's Flow Charts – CreateInverseGaussianFilter



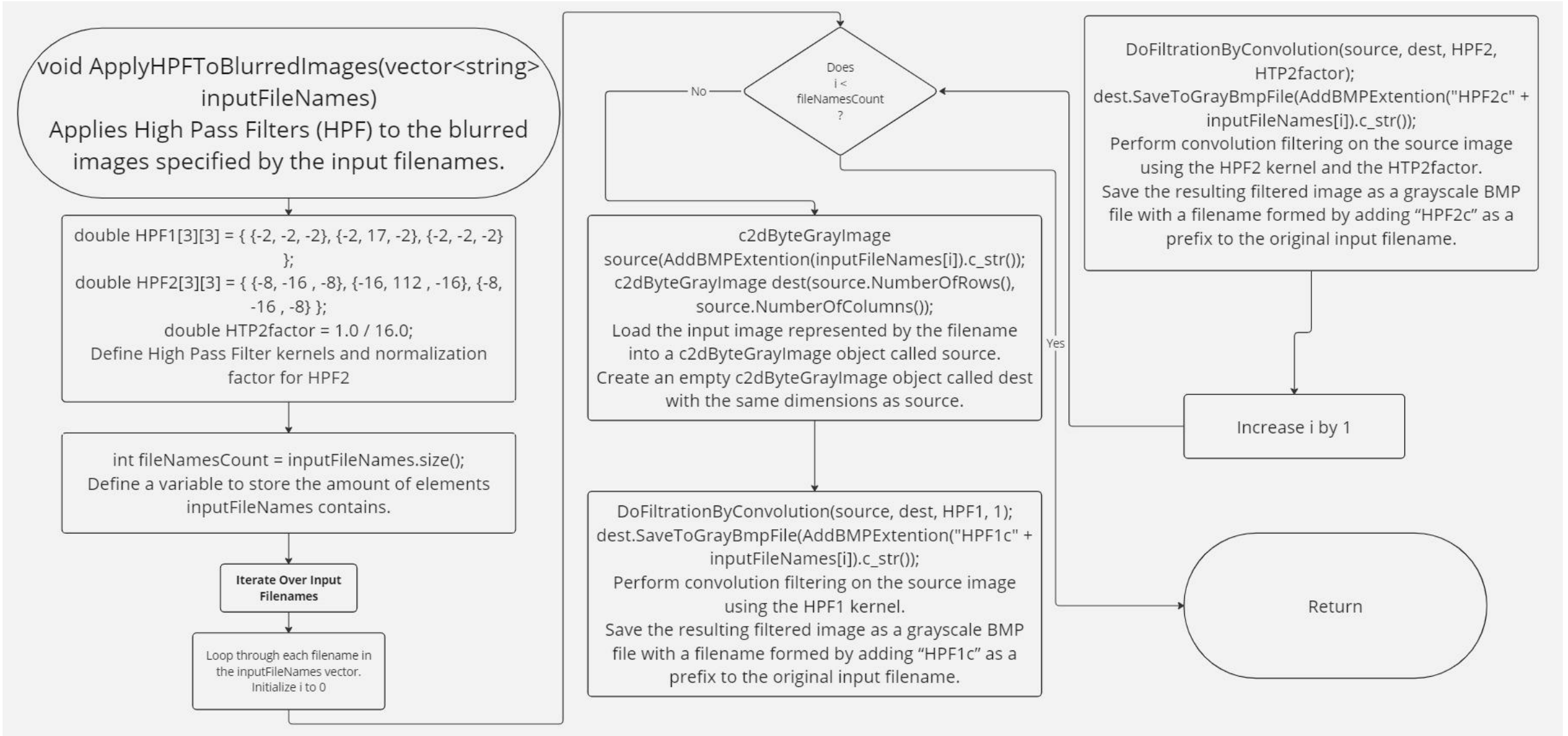
MP2.4. Algorithm's Flow Charts – ApplyFFT



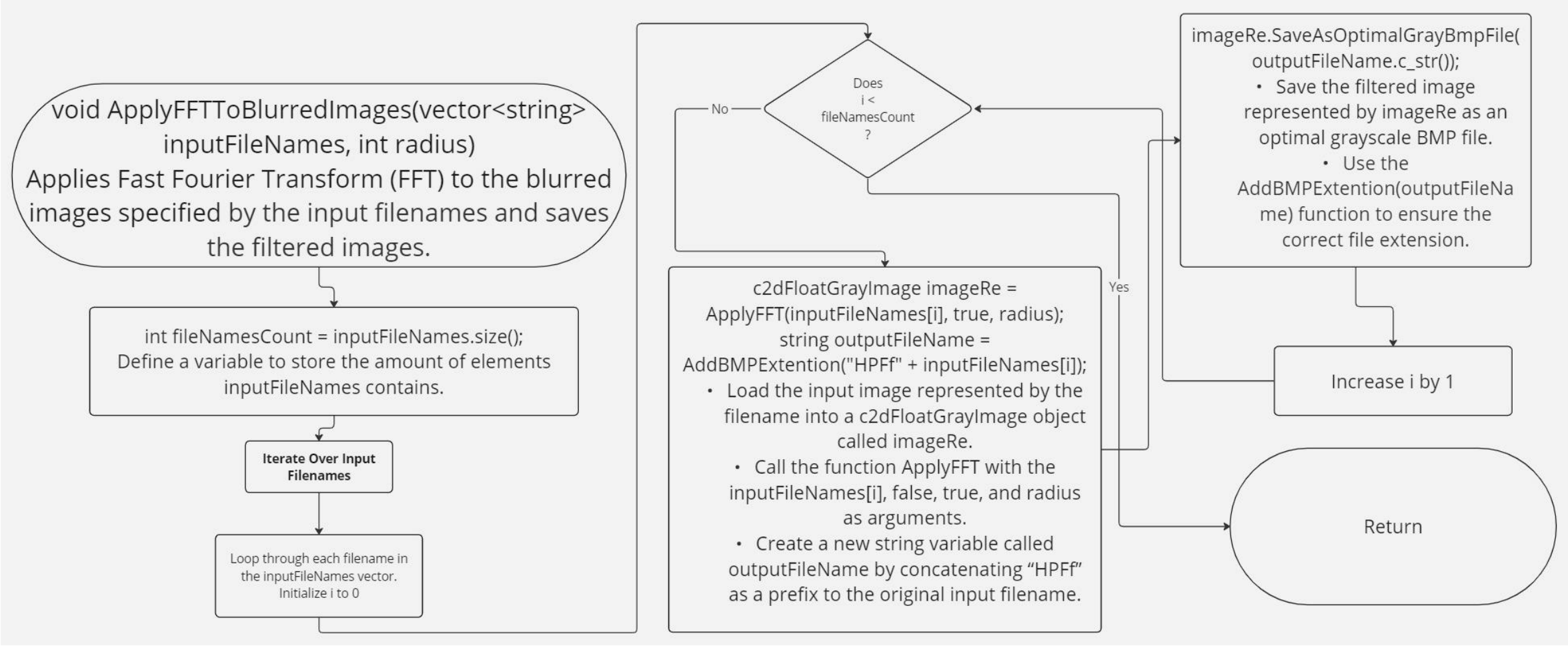
MP2.4. Algorithm's Flow Charts – ApplyFastGaussianFilterConvolution



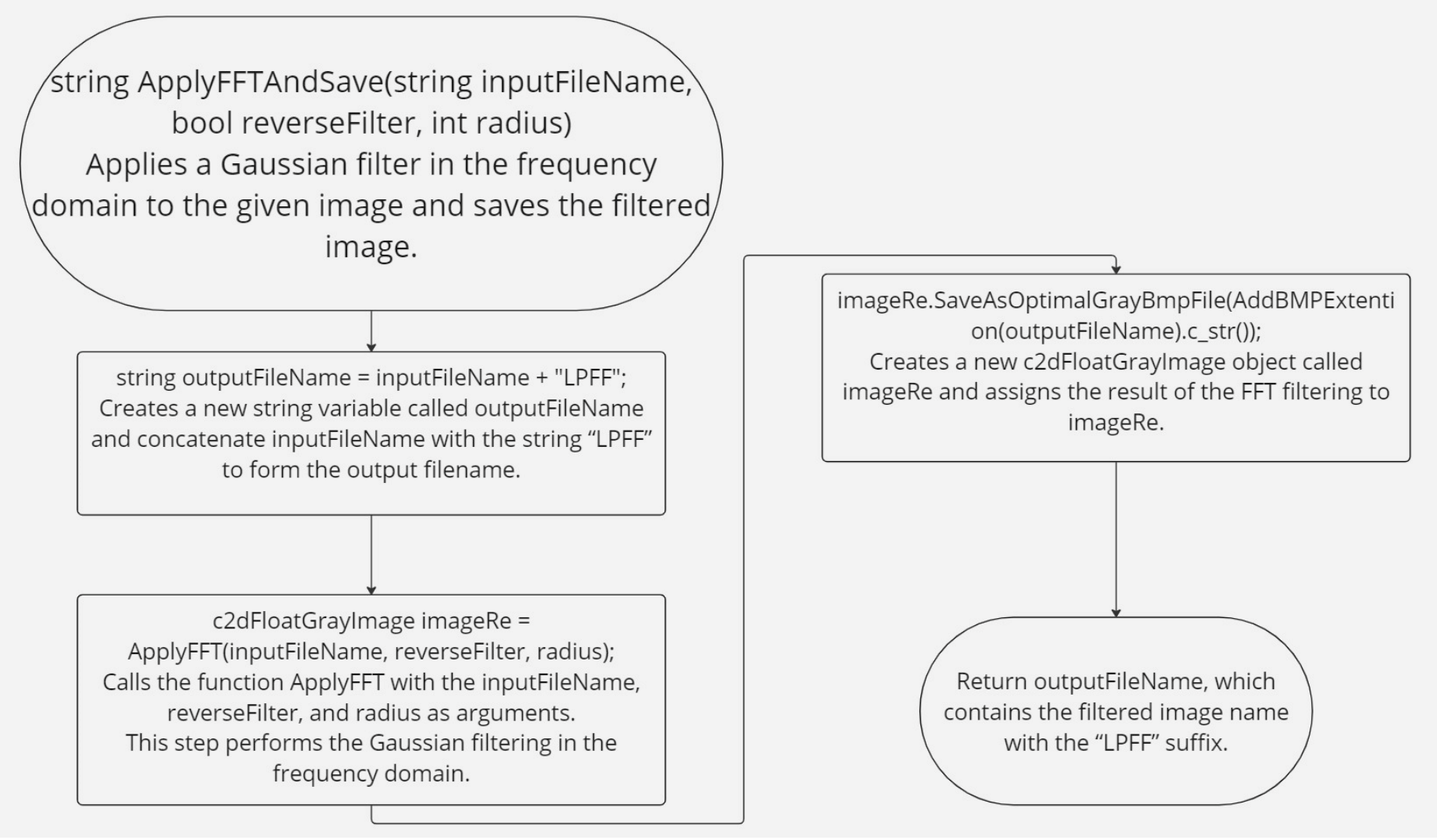
MP2.4. Algorithm's Flow Charts – ApplyHPFToBlurredImages



MP2.4. Algorithm's Flow Charts – ApplyFFTToBlurredImages



MP2.4. Algorithm's Flow Charts – ApplyFFTAndSave



MP2.5. What Did We Learn

- How to design different filters to smoothen an image.
- How to work in spatial domain and frequency domain.
- Usage convolution filters.
- Usage of FFT and it’s filters.
- How to use the FFT library.
- How to create different filters to sharpen an image using HPF by convolution.
- How to restore the sharpness of blurry images using HPF by FFT.
- Each image might require a different HPF to achieve the best results in sharpening/restoring it.

MP2.6. List of Sources - Bibliography

1. R. Fisher, S. Perkins, A. Walker and E. Wolfart. (2003). Gaussian Smoothing. (accessed: 19.04.2024) https://homepages.inf.ed.ac.uk/rbf/HIPR2/gsmooth.htm
2. Weinhaus ,F. Digital Image Filtering. (accessed: 19.04.2024) http://www.fmwconcepts.com/imagemagick/digital_image_filtering.pdf
3. Dr. Kosolapov, S. Lectures 08 Filtration by Convolution, 09 FFT Part 1, 10 FFT Part 2 (accessed: 16.04.2024) https://sites.google.com/view/improc31651
4. Graus, C. (2005). Image Processing for Dummies with C# and GDI+ Part 2 - Convolution Filters (accessed: 21.04.2024) https://www.codeproject.com/Articles/2008/Image-Processing-for-Dummies-with-C-and-GDI-Part-2